BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐỒNG NAI



BÁO CÁO CUỐI KỲ

LẬP TRÌNH MOBILE

ĐỀ TÀI:

ỨNG DỤNG QUẢN LÝ CHI TIÊU CÁ NHÂN

Giảng viên hướng dẫn: ThS. Huỳnh Hoàng Tân

Nhóm sinh viên thực hiện:

- Trịnh Thành Đạt – 1923050149
- Trần Trọng Đại – 1923050745

ĐỒNG NAI – THÁNG 10 NĂM 2026

BẢNG PHÂN CÔNG NHIỆM VỤ VÀ TỶ LỆ ĐÓNG GÓP

STT	Họ và Tên	Vai Trò & Trách Nhiệm Đảm Nhận	Đóng Góp	Đánh Giá
1	Trịnh Thành Đạt (1923050149)	Trưởng nhóm, Lead Developer & System Architect: - Thiết kế kiến trúc tổng thể Clean Architecture cho Backend ASP.NET Core 10 Web API & CSDL PostgreSQL 16 3NF. - Hiện thực hóa toàn bộ 22 REST Controllers, Realtime FinancialHub (/hubs/financial), FinancialRealtimeNotifier, cơ chế khóa bi quan (Pessimistic Locking FOR UPDATE) chống Race-Condition số dư ví và Deadlock Prevention. - Tích hợp Gateway Google Gemini AI, thuật toán Hậu kiểm toán số học OCR và Jev Action Protocol. - Xây dựng bộ kịch bản nạp dữ liệu Demo Story và phát triển bộ script kiểm thử tự động 113 API Endpoints (test_api_suite.ps1). - Thiết lập kiến trúc Clean MVVM cho Frontend .NET MAUI: Model Repositories (hỗ trợ Mock/Api 0ms), ViewModels, Refit Client, SignalRRealtimeService và gắn 100% Data-Binding cho 20 màn hình.	75%	Hoàn thành 100%
2	Trần Trọng Đại (1923050745)	UI/UX Designer & Frontend Specialist: - Phụ trách toàn diện Giai đoạn 9 (Dedicated Frontend Art Direction & UI/UX Styling) theo cẩm nang kỹ thuật HUONG_DAN_FE.md. - Tiếp quản 20 Wireframe Views XAML, thiết lập Design System tập trung tại Resources/Styles (Bảng màu Slate & Charcoal, kiểu chữ, icons). - Tinh chỉnh bố cục giao diện người dùng, khoảng cách padding/margin chuẩn 8pt, hỗ trợ hiển thị tối ưu trên cả Windows Desktop và Android Mobile. - Thiết kế trạng thái rỗng (Empty States), biểu tượng trạng thái đang tải (Loading), thông báo lỗi và hoàn thiện trải nghiệm thị giác người dùng.	25%	Hoàn thành 100%

MỤC LỤC

MỤC LỤC 2
ĐẶT VẤN ĐỀ BÀI TOÁN 4
1. Bối cảnh thực tiễn và lý do chọn đề tài 4
2. Mục tiêu nghiên cứu và giải pháp đề xuất 4
CÁC CHỨC NĂNG CHÍNH CỦA ĐỀ TÀI 5
CHƯƠNG 1: THIẾT KẾ CƠ SỞ DỮ LIỆU VÀ XÂY DỰNG BACKEND 6
1.1 Mô hình cơ sở dữ liệu quan hệ chuẩn 3NF (PostgreSQL 16) 6
1.2 Xây dựng hệ thống dịch vụ ASP.NET Core 10 Web API 8
1.3 Trích đoạn mã nguồn xử lý logic cốt lõi phía Backend 13
1.4 Cổng truyền thông thời gian thực SignalR Gateway và Push Notification 15
CHƯƠNG 2: THIẾT KẾ VÀ XÂY DỰNG GIAO DIỆN ỨNG DỤNG DI ĐỘNG BẰNG .NET MAUI 16
2.1 Kiến trúc ứng dụng di động toàn diện .NET MAUI 16
2.2 Ứng dụng mô hình MVVM và CommunityToolkit.Mvvm 16
2.2.1 Giải thích luồng dữ liệu kiến trúc MVVM cho chức năng mẫu 19
2.2.2 Báo cáo kết quả kiểm thử tự động (Unit Testing) 20
2.3 Giao diện thực thi thực tế của ứng dụng di động 21
2.4 Cơ chế cập nhật dữ liệu động thời gian thực và thông báo đẩy 32
2.5 Khả năng vận hành ngoại tuyến (Offline-First) và Mock Repository 34
CHƯƠNG 3: KẾT LUẬN VÀ ĐỊNH HƯỚNG 35
3.1 Đánh giá Kết quả Đạt được 35
3.2 Khó khăn trong quá trình phát triển 35
3.2.1 Thách thức về cơ chế quản trị tiến trình nền và hệ thống thông báo trên hệ điều hành di động (Android Lifecycle & Background Restrictions) 35

3.2.2 Thách thức về đồng bộ dữ liệu thời gian thực đa thiết bị (Real-time Concurrency & SignalR Synchronization) 36
3.2.3 Thách thức về kiến trúc dữ liệu ngoại tuyến (Offline-First Architecture & Dual Repository) 36
3.2.4 Thách thức về chuẩn hóa giao diện đa nền tảng và tuân thủ mô hình MVVM thuần túy 37
3.3 Định hướng Phát triển 37

ĐẶT VẤN ĐỀ BÀI TOÁN

1. Bối cảnh thực tiễn và lý do chọn đề tài

Trong bối cảnh nền kinh tế số và xu hướng thanh toán không dùng tiền mặt phát triển bùng nổ, việc quản lý tài chính cá nhân (Personal Finance Management - PFM) đã trở thành bài toán cấp thiết đối với mỗi cá nhân, đặc biệt là thế hệ trẻ, sinh viên và nhân sự văn phòng. Sự thuận tiện của quét mã VietQR, chuyển khoản tức thì và quẹt thẻ tín dụng vô tình làm suy giảm 'cảm giác đau khi chi tiền' (Pain of Paying), thúc đẩy hành vi mua sắm bốc đồng và dẫn tới nguy cơ thâm hụt tài chính nghiêm trọng trước mỗi kỳ nhận lương.

Khảo sát các ứng dụng quản lý chi tiêu hiện có trên thị trường cho thấy đa phần vẫn vận hành theo phương thức ghi chép thủ công: người dùng phải tự mở ứng dụng, gõ từng con số, chọn từng danh mục sau mỗi lần mua sắm. Rào cản nhận thức và sự tốn kém thời gian này khiến trên 70% người dùng bỏ cuộc chỉ sau vài tuần sử dụng. Bên cạnh đó, các ứng dụng hiện hành thiếu cơ chế đồng bộ biến động dữ liệu thời gian thực (Realtime Synchronization), hạn chế trong việc vận hành khi mất kết nối mạng (Offline-First) và hoàn toàn vắng bóng các cú hích hành vi thông minh (Nudge Alerts) cùng thông báo đẩy thực sự (Native Push Notifications) tại thời điểm phát sinh rủi ro tài chính.

2. Mục tiêu nghiên cứu và giải pháp đề xuất

Đề tài 'Hệ thống Quản lý Chi tiêu và Hoạch định Tài chính Cá nhân Thông minh' được xây dựng nhằm giải quyết triệt để các hạn chế trên thông qua việc ứng dụng hệ sinh thái công nghệ di động hiện đại Microsoft .NET MAUI 10 (LTS) kết hợp kiến trúc dịch vụ phân tán ASP.NET Core 10 Web API và CSDL PostgreSQL 16. Hệ thống hướng đến mục tiêu: tự động hóa tối đa khâu ghi nhận chi tiêu qua thị giác AI OCR, duy trì kết nối thời gian thực hai chiều SignalR 0ms, đảm bảo khả năng hoạt động liên tục khi ngoại tuyến nhờ cơ sở dữ liệu cục bộ SQLite, đồng thời tích hợp hệ thống thông báo đẩy chuẩn thiết bị di động (Native Mobile Push Notifications) nhằm thúc đẩy kỷ luật tài chính bền vững.

CÁC CHỨC NĂNG CHÍNH CỦA ĐỀ TÀI

Dựa trên các tiêu chí chuẩn mực của đồ án môn học Lập trình Di động và yêu cầu nghiệp vụ quản lý tài chính cá nhân, hệ thống được quy hoạch thành 6 khối chức năng cốt lõi sau:

Quản trị Đa Tài khoản & Sổ cái Bất biến (Multi-Wallet ACID): Hỗ trợ theo dõi đồng thời nhiều nguồn tiền: Tiền mặt, Tài khoản Ngân hàng (VCB, ACB, MB) và Thẻ tín dụng. Hỗ trợ chuyển tiền nội bộ an toàn và lưu vết lịch sử biến động số dư bất biến.

Quản lý Thu nhập & Chi tiêu Cá nhân: Ghi nhận giao dịch thu chi nhanh chóng với cây danh mục phân cấp (Ăn uống, Nhà ở, Đi lại, Mua sắm...). Tự động tính toán tổng thu, tổng chi và số dư tài chính theo thời gian thực.

Hoạch định Ngân sách Thích ứng (DSA) & Cảnh báo Hạn mức: Thiết lập ngân sách theo quy tắc 50/30/20 hoặc Kakeibo. Tự động tính toán Hạn mức Chi tiêu An toàn Mỗi Ngày (Daily Safe Spending Allowance - DSA) và cảnh báo khi tốc độ tiêu tiền vượt ngưỡng.

Quản lý Mục tiêu Tích lũy & Mô hình 6 Chiếc Hũ (Jars): Thiết lập các quỹ mục tiêu tài chính (Mua laptop, Quỹ phòng thân 6 tháng). Hỗ trợ nộp tiền, rút tiền và phân bổ quỹ tự động theo phương pháp 6 Chiếc Hũ.

Cập nhật Dữ liệu Động Thời Gian Thực (Realtime SignalR 0ms): Tích hợp kết nối WebSocket Full-Duplex qua ASP.NET Core SignalR Hub (/hubs/financial). Mọi biến động số dư hoặc giao dịch mới lập tức phản ánh đồng thời trên các thiết bị đang đăng nhập mà không cần tải lại trang.

Thông báo Đẩy Thiết Bị Thật (Native Mobile Push Notifications): Kích hoạt thông báo nổi (Heads-Up Banner) và lưu trữ lịch sử thông báo trên khay hệ điều hành khi phát sinh cảnh báo vượt ngân sách, khoản chi cần khoảng trễ 24h hoặc thông báo nhắc lịch chi tiêu định kỳ.

Tiện ích Tài chính Thông minh (AI OCR & VietQR Napas 247): Bóc tách hóa đơn mua sắm tự động bằng Multimodal AI OCR kèm thuật toán hậu kiểm toán số học; tạo mã thanh toán VietQR Napas 247 tức thời theo chuẩn ngân hàng quốc gia.

CHƯƠNG 1: THIẾT KẾ CƠ SỞ DỮ LIỆU VÀ XÂY DỰNG BACKEND

1.1 Mô hình cơ sở dữ liệu quan hệ chuẩn 3NF (PostgreSQL 16)

Cơ sở dữ liệu của hệ thống được xây dựng trên hệ quản trị PostgreSQL 16 (triển khai trên Docker Container), được chuẩn hóa nghiêm ngặt ở dạng chuẩn 3 (3NF) nhằm loại bỏ hoàn toàn các dị thường thêm, xóa, sửa và bảo toàn tính toàn vẹn tham chiếu tài chính thông qua khóa ngoại liên kết giữa các bảng. Bảng 1.1 liệt kê danh mục 22 bảng thực thể cốt lõi:

Bảng 1.1: Danh mục các bảng thực thể cốt lõi trong Cơ sở Dữ liệu PostgreSQL 16

Tên Bảng	Khóa Chính	Khóa Ngoại	Mục Đích Sử Dụng Nghiệp Vụ
users	id (UUID)	Không có	Lưu trữ tài khoản, họ tên, email đăng nhập và mật khẩu băm BCrypt an toàn.
wallets	id (UUID)	user_id -> users(id)	Quản lý danh sách các ví (Tiền mặt, Ngân hàng, Thẻ tín dụng) và số dư khả dụng.
wallet_transfers	id (UUID)	source/dest -> wallets(id)	Nhật ký lịch sử chuyển tiền nội bộ giữa các tài khoản ví.
categories	id (UUID)	parent_id -> categories(id)	Cây danh mục thu chi phân cấp, gắn nhãn mô hình 50/30/20 và Kakeibo.
transactions	id (UUID)	wallet_id, category_id	Ghi nhận giao dịch thu - chi tài chính, tự động khấu trừ/cộng dồn số dư ví.
receipt_items	id (UUID)	transaction_id	Lưu danh sách các món hàng con bóc tách từ hóa đơn mua sắm bán lẻ.

recurring_incomes	id (UUID)	wallet_id, category_id	Lịch thu nhập định kỳ (Lương, Freelance) và cơ chế tự động ghi sổ khi đến hạn.
budgets	id (UUID)	user_id -> users(id)	Kế hoạch ngân sách theo kỳ (Tháng) và định mức chi tiêu an toàn.
budget_allocations	id (UUID)	budget_id, category_id	Phân bổ hạn mức chi tiêu chi tiết cho từng danh mục riêng biệt.
savings_goals	id (UUID)	user_id -> users(id)	Mục tiêu tích lũy tài chính, quỹ phòng thân và mô hình 6 chiếc hũ.
savings_goal_contributions	id (UUID)	savings_goal_id, wallet_id	Lịch sử các lần nộp tiền hoặc rút tiền khỏi quỹ tích lũy.
financial_nudges	id (UUID)	user_id -> users(id)	Lưu trữ các Cú hích hành vi thông minh (Nudges) và khuyến nghị tài chính.
cooling_off_transactions	id (UUID)	user_id, wallet_id	Giao dịch chi tiêu mua sắm xa xỉ tạm hoãn 24h chờ duyệt lý trí.
user_otp_codes	id (UUID)	user_id -> users(id)	Quản lý mã xác thực OTP 6 chữ số, thời hạn hiệu lực và số lần thử.
user_profiles	id (UUID)	user_id -> users(id)	Hồ sơ người dùng, ảnh đại diện, đơn vị tiền tệ mặc định (VND).
auto_categorization_rules	id (UUID)	user_id -> users(id)	Quy tắc tự động gán danh mục dựa trên từ khóa và biểu thức chính quy.

salary_slip_details	id (UUID)	recurring_income_id	Chi tiết bóc tách phiếu lương AI OCR (Gross, BHXH, BHYT, Thuế TNCN, Net).
in_app_notifications	id (UUID)	user_id -> users(id)	Lưu trữ lịch sử thông báo nội bộ, phân loại cảnh báo và cờ đã đọc.
device_push_tokens	id (UUID)	user_id -> users(id)	Quản lý Push Token thiết bị di động Android gắn với tài khoản người dùng.
autonomous_actions	id (UUID)	user_id -> users(id)	Hành động tài chính tự hành sinh ra từ bóc tách thông báo ngân hàng thông qua mô phỏng Jev.
spending_forecast_snapshots	id (UUID)	user_id -> users(id)	Ảnh chụp dự báo chuỗi thời gian chi tiêu cuối kỳ và Ngày cạn tiền khả dụng.
what_if_simulations	id (UUID)	user_id -> users(id)	Kịch bản mô phỏng giả định What-If Sandbox tài chính cá nhân.

1.2 Xây dựng hệ thống dịch vụ ASP.NET Core 10 Web API

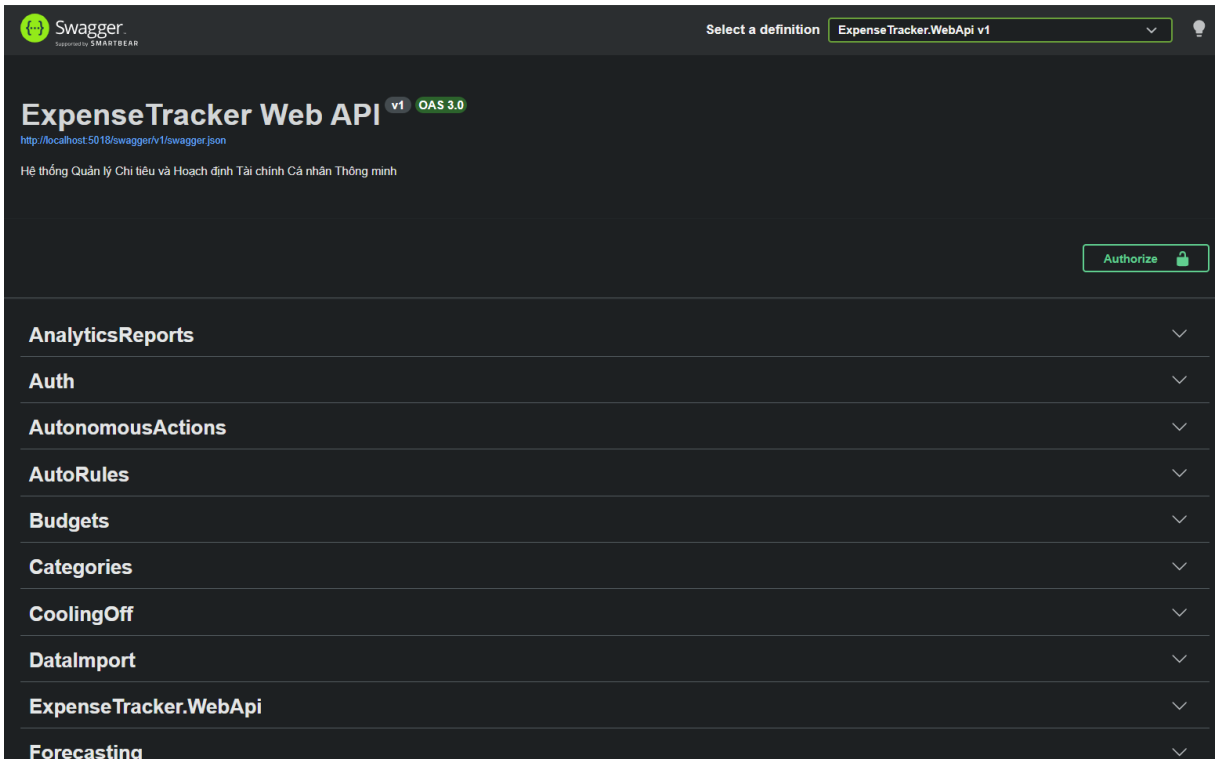
Phân hệ Backend được xây dựng trên nền tảng ASP.NET Core 10 Web API kết hợp Entity Framework Core 10, áp dụng kiến trúc Clean Architecture phân rã thành 4 tầng rành mạch: ExpenseTracker.Domain (Thực thể miền & Quy tắc nghiệp vụ), ExpenseTracker.Application (Use Cases, DTOs & Validation), ExpenseTracker.Infrastructure (Truy cập dữ liệu PostgreSQL, SignalR & Push Adapter) và ExpenseTracker.WebApi (Cung cấp 22 RESTful Controllers & Auth Middleware). Bảng 1.2 tóm tắt các endpoint chính:

Bảng 1.2: Danh mục các nhóm API RESTful Backend ASP.NET Core phục vụ phân hệ di động

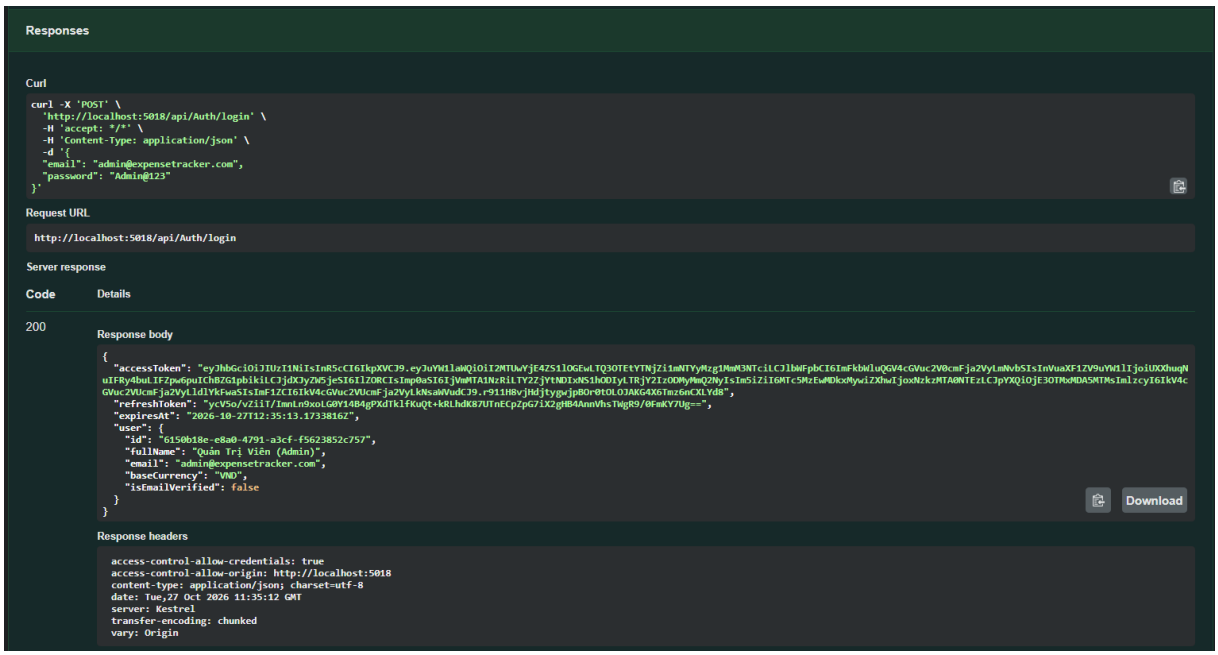
Phương Thức	Đường Dẫn API (Endpoint)	Chức Năng Nghiệp Vụ Cốt Lõi
POST	/api/auth/register, /api/auth/login	Xác thực tài khoản, cấp phát cặp JWT Access Token và Refresh Token.
GET, PUT	/api/users/profile	Xem và cập nhật thông tin hồ sơ người dùng, đơn vị tiền tệ mặc định.
GET, POST	/api/wallets	Xem danh sách các ví tài chính và tạo ví mới (Tiền mặt, Ngân hàng, Thẻ tín dụng).
POST	/api/wallets/transfer	Chuyển tiền nội bộ giữa 2 ví đảm bảo tính toàn vẹn số dư ACID.
POST	/api/wallets/{id}/adjust	Cân chỉnh số dư thực tế kèm nhật ký kiểm kê và lý do điều chỉnh.
GET, POST	/api/categories	Xem cây danh mục thu - chi phân cấp và tạo danh mục tùy chỉnh cá nhân.
POST	/api/transactions/incomes	Ghi nhận khoản thu nhập, tự động cộng dồn số dư vào ví đích.
POST	/api/transactions/expenses	Ghi nhận khoản chi tiêu, tự động khấu trừ số dư ví kèm danh sách món hàng.
GET	/api/transactions	Truy vấn lịch sử giao dịch có phân trang, lọc theo thời gian, ví và danh mục.
GET, POST	/api/budgets	Lập kế hoạch ngân sách tháng và theo dõi tiến độ chi tiêu theo thời gian thực.
GET	/api/budgets/active/dsa	Tính toán hạn mức chi tiêu an toàn mỗi ngày (Daily Safe Spending Allowance).
GET, POST	/api/savings-goals	Tạo và quản lý mục tiêu tích lũy tài chính,

		phân bổ mô hình 6 chiếc hũ.
GET, POST	/api/notifications	Truy vấn và đánh dấu đã đọc danh sách thông báo in-app.
GET, POST	/api/cooling-off	Khởi tạo khoảng trễ tâm lý 24h đối với khoản mua sắm bốc đồng vượt ngưỡng.
POST	/api/smart-utilities/vietqr	Sinh mã QR Napas 247 tức thời phục vụ thanh toán chuyển khoản.
POST	/api/system/reset-database	Dọn dẹp và khôi phục CSDL về trạng thái kịch bản Demo hoàn hảo.

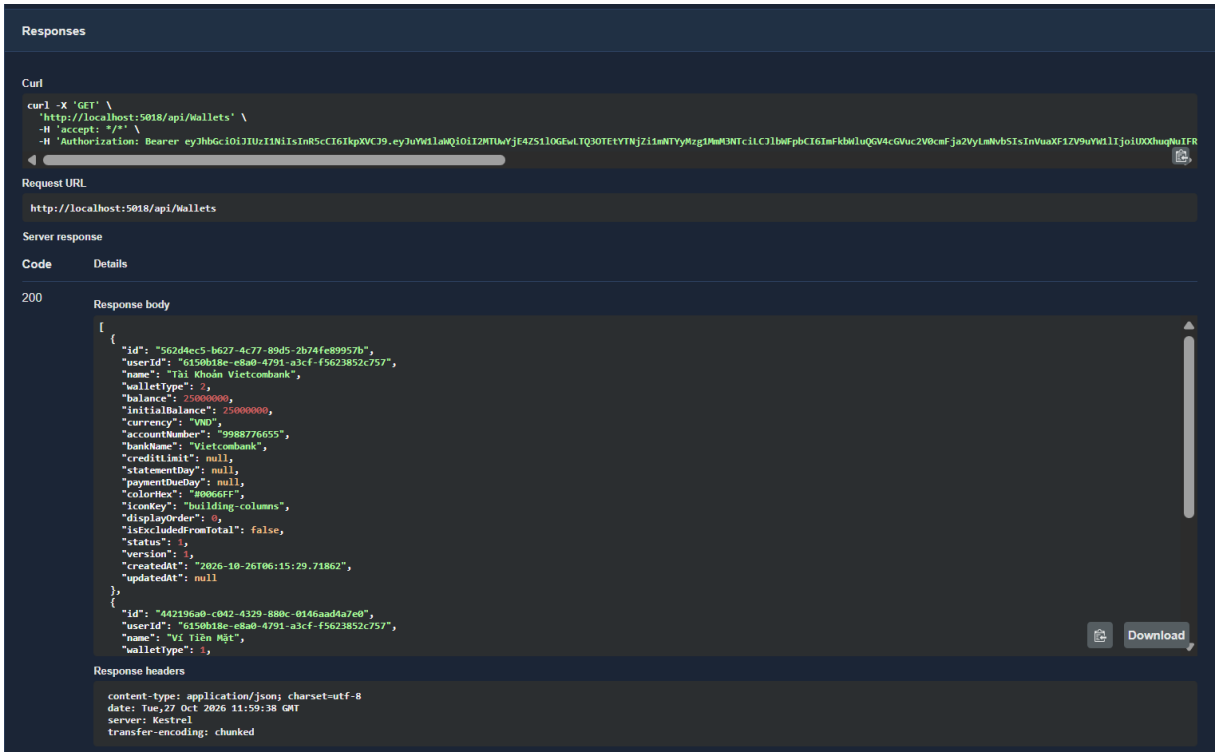
Dưới đây là hình ảnh thực thi thực tế của các API cốt lõi được kiểm thử trực tiếp trên Swagger UI kết hợp kết quả kiểm thử tự động toàn diện trên bộ kịch bản test:



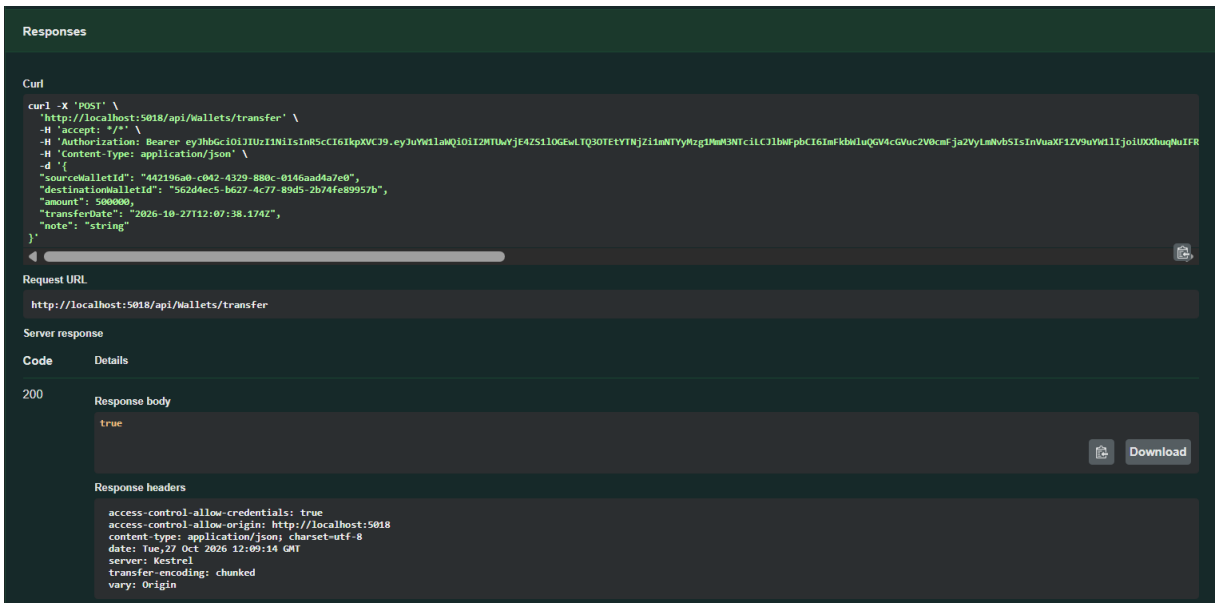
Hình 1. 1 Giao diện tổng quan tài liệu Swagger UI của ASP.NET Core 10 Web API



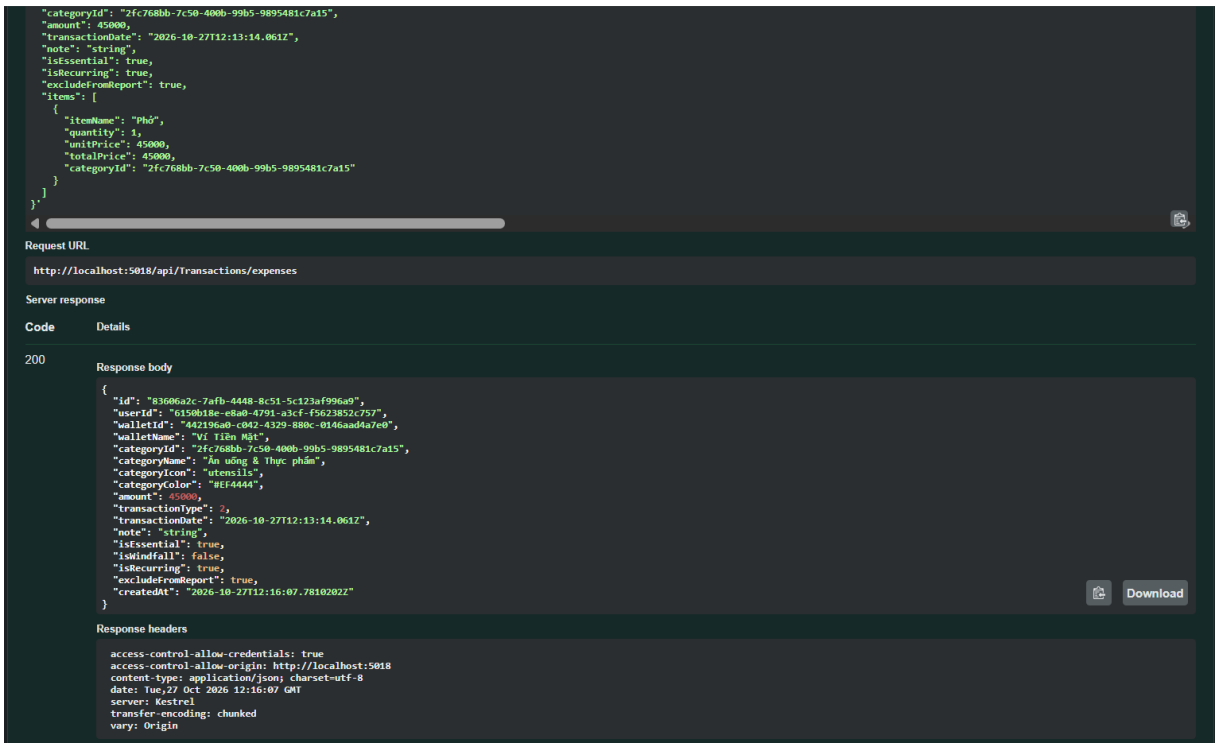
Hình 1. 2 Kết quả thực thi API Xác thực Đăng nhập (/api/auth/login) trên Swagger UI



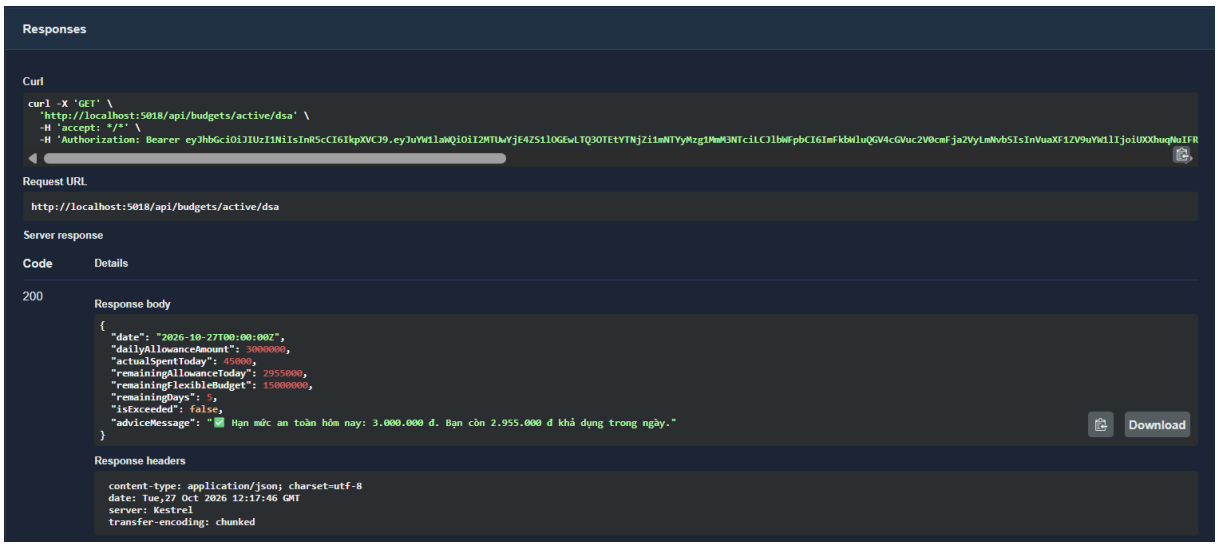
Hình 1. 3 Kết quả thực thi API Quản lý Ví và Số dư (/api/wallets) trên Swagger UI



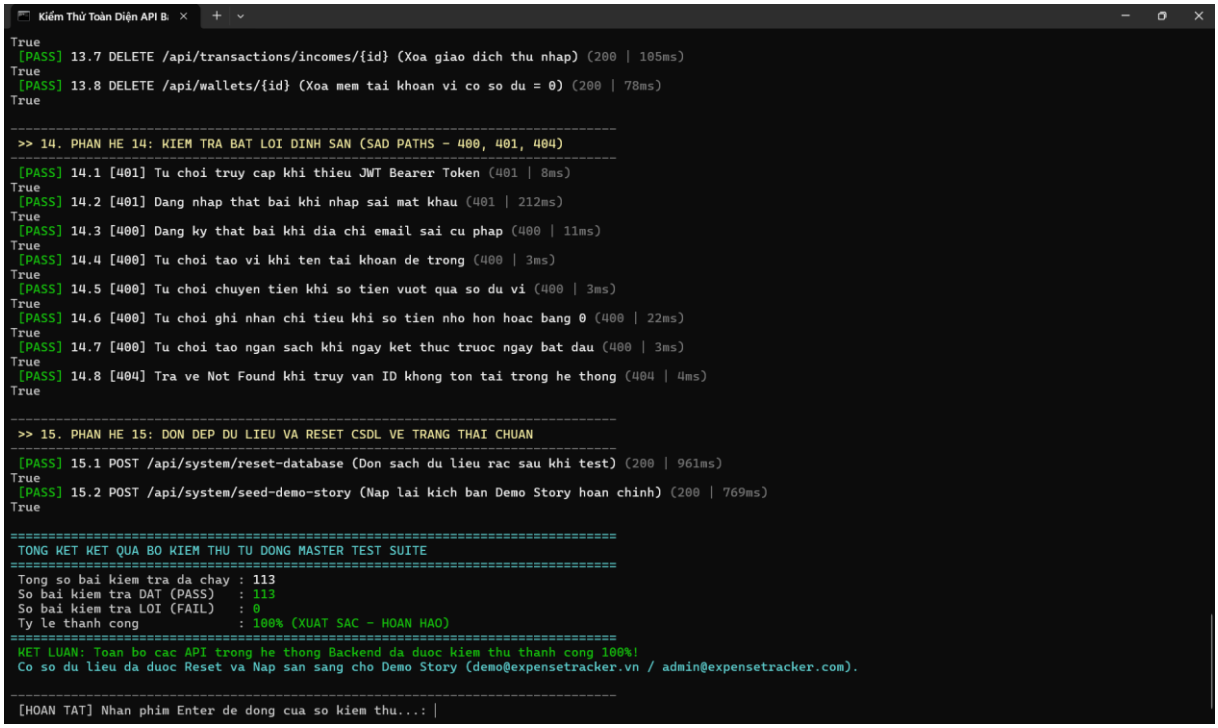
Hình 1. 4 Kết quả thực thi API Chuyển tiền nội bộ giữa các ví (/api/wallets/transfer) trên Swagger UI



Hình 1. 5 Kết quả thực thi API Ghi nhận Giao dịch Chi tiêu (/api/transactions/expenses) trên Swagger UI



Hình 1. 6 Kết quả thực thi API Hoạch định Ngân sách và Hạn mức DSA (/api/budgets/active/dsa) trên Swagger UI



Hình 1. 7 Kết quả chạy tự động 113 API Tests đạt 100% PASS trên PowerShell Console

1.3 Trích đoạn mã nguồn xử lý logic cốt lõi phía Backend

Để bảo đảm tính toàn vẹn số liệu tài chính tuyệt đối và loại trừ triệt để tình trạng Race-Condition khi có nhiều giao dịch hoặc chuyển tiền phát sinh đồng thời, hệ thống áp dụng cơ chế Khóa Bi Quan (Pessimistic Locking) trực tiếp trên tầng CSDL PostgreSQL bằng câu lệnh SELECT ... FOR UPDATE thông qua ApplicationDbContext:

```
// Mã nguồn minh họa (ApplicationDbContext.cs - Khóa Bi Quan FOR UPDATE):
```

```
public async Task<Wallet?> GetWalletForUpdateAsync(Guid walletId, Guid
userId, CancellationToken ct = default)
{
    if (Database.ProviderName?.Contains("Npgsql",
StringComparison.OrdinalIgnoreCase) == true)
    {
        return await Wallets
            .FromSqlRaw("SELECT * FROM \"wallets\" WHERE \"Id\" = {0} AND
\"UserId\" = {1} AND \"IsDeleted\" = false FOR UPDATE",
                        walletId, userId)
            .FirstOrDefaultAsync(ct);
    }
    return await Wallets.FirstOrDefaultAsync(w => w.Id == walletId &&
w.UserId == userId, ct);
}
```

Đoạn mã sau trong WalletService minh họa giải thuật Chuyển tiền nội bộ nguyên tử (ACID Transaction) kết hợp cơ chế phòng chống Deadlock bằng cách sắp xếp thứ tự GUID trước khi khóa tài nguyên và phát sóng Realtime:

// Mã nguồn minh họa (WalletService.cs - Chuyển Tiền An Toàn ACID & Chống Deadlock):

```
public async Task<Result<bool>> TransferMoneyAsync(Guid userId,
TransferMoneyRequest request, CancellationToken ct = default)
{
    if (request.SourceWalletId == request.DestinationWalletId)
        return Result<bool>.Failure("Ví nguồn và ví đích không được trùng
nhau.");

    await using var transaction = await _context.BeginTransactionAsync(ct);
    try {
        bool sourceIsFirst =
request.SourceWalletId.CompareTo(request.DestinationWalletId) < 0;
        var firstLockId = sourceIsFirst ? request.SourceWalletId :
request.DestinationWalletId;
        var secondLockId = sourceIsFirst ? request.DestinationWalletId :
request.SourceWalletId;

        var firstWallet = await
_context.GetWalletForUpdateAsync(firstLockId, userId, ct);
        var secondWallet = await
_context.GetWalletForUpdateAsync(secondLockId, userId, ct);
        var sourceWallet = sourceIsFirst ? firstWallet : secondWallet;
        var destWallet = sourceIsFirst ? secondWallet : firstWallet;

        sourceWallet.Debit(request.Amount);
        destWallet.Credit(request.Amount);

        var transfer = new WalletTransfer(userId, sourceWallet.Id,
destWallet.Id, request.Amount, request.TransferDate, request.Note);
        await _context.WalletTransfers.AddAsync(transfer, ct);

        await _context.SaveChangesAsync(ct);
        await transaction.CommitAsync(ct);
```

```
            return Result<bool>.Success(true);
        }
        catch (Exception) { await transaction.RollbackAsync(ct); throw; }
    }
```

1.4 Cổng truyền thông thời gian thực SignalR Gateway và Push Notification

Hệ thống tích hợp ASP.NET Core SignalR Hub tại địa chỉ endpoint '/hubs/financial', thiết lập kênh truyền thông 2 chiều Full-Duplex dựa trên giao thức WebSocket chuẩn. Hub được bảo vệ nghiêm ngặt bằng JWT Authentication (thông qua query string access_token khi bắt tay). Mỗi khi có giao dịch thu chi mới hoặc thay đổi số dư ví, máy chủ lập tức phát sóng sự kiện 'ReceiveBalanceUpdate' đến đúng định danh tài khoản người dùng thông qua nhóm 'user_{userId}', bảo đảm giao diện phía Client cập nhật tức thì 0ms:



Hình 1. 8 Cổng kết nối thời gian thực SignalR Hub (/hubs/financial) trên ASP.NET Core Web API

```
// Mã nguồn minh họa (FinancialHub.cs & FinancialRealtimeNotifier.cs - Realtime
Broadcast):

[Authorize]
public class FinancialHub : Hub
{
    public override async Task OnConnectedAsync()
    {
        var userId = GetUserId();
        if (userId.HasValue) {
            var userGroup = $"user_{userId.Value}";
            await Groups.AddToGroupAsync(Context.ConnectionId, userGroup);
        }
        await base.OnConnectedAsync();
    }
}

public async Task NotifyBalanceUpdateAsync(Guid userId,
BalanceUpdateNotificationDto dto, CancellationToken ct = default)
{
    var groupName = $"user_{userId}";
    await
_hubContext.Clients.Group(groupName).SendAsync("ReceiveBalanceUpdate", dto,
ct);
}
```

CHƯƠNG 2: THIẾT KẾ VÀ XÂY DỰNG GIAO DIỆN ỨNG DỤNG DI ĐỘNG BẰNG .NET MAUI

2.1 Kiến trúc ứng dụng di động toàn diện .NET MAUI

Ứng dụng di động được xây dựng trên nền tảng Microsoft .NET MAUI 10 (LTS) sử dụng ngôn ngữ C# và XAML, hỗ trợ triển khai đa nền tảng cho cả Windows Desktop và Android Mobile. Toàn bộ mã nguồn phía Client tuân thủ nguyên tắc kiến trúc phân lớp sạch, tách biệt hoàn toàn giữa giao diện hiển thị, trạng thái trình bày và kết nối mạng.

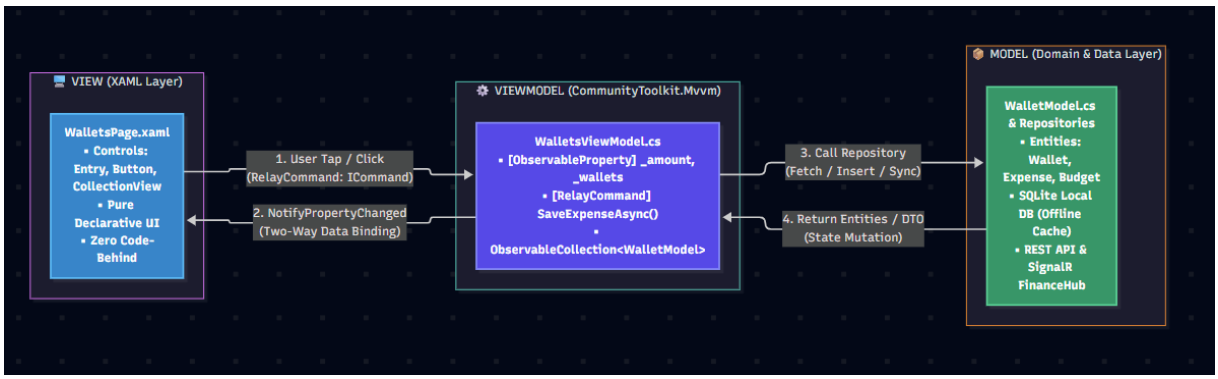
2.2 Ứng dụng mô hình MVVM và CommunityToolkit.Mvvm

Dự án áp dụng triệt để mô hình kiến trúc MVVM (Model - View - ViewModel) kết hợp thư viện CommunityToolkit.Mvvm hiện đại. Quy chuẩn kỹ thuật loại bỏ 100% mã xử lý logic trong tệp giao diện (Zero Code-Behind trong các tệp .xaml.cs). Mọi tương tác của người dùng, luồng tải dữ liệu và kiểm tra ràng buộc đều được đóng gói an toàn trong ViewModel thông qua cơ chế Data-Binding:

Tầng Model: Bao gồm các thực thể dữ liệu, DTOs và các Repositories kế thừa Interface chung (IWalletRepository, ITransactionRepository...). Hỗ trợ hoán đổi linh hoạt giữa MockRepository (chứa dữ liệu mẫu chạy offline) và ApiRepository (kết nối máy chủ thật qua Refit REST Client).

Tầng ViewModel: Kế thừa từ BaseViewModel, sử dụng thuộc tính [ObservableProperty] để tự động kích hoạt thông báo INotifyPropertyChanged và [RelayCommand] để đóng gói hành vi của nút bấm (Thêm giao dịch, Chuyển tiền, Lọc ngày).

Tầng View: Hệ thống 20 màn hình giao diện XAML thuần túy, chỉ chứa các mỏ neo liên kết dữ liệu {Binding ...} và tuyệt đối không trực tiếp gọi API hay truy xuất bộ nhớ cục bộ.



Hình 2. 1 Sơ đồ Kiến trúc Clean MVVM 3 tầng trong ứng dụng .NET MAUI

Bảng 2.1: Danh mục các ViewModels và Views tương ứng trong ứng dụng .NET MAUI

Tên Màn Hình (View)	Lớp ViewModel Quản Lý	Vai Trò & Chức Năng Nghiệp Vụ
LoginPage.xaml	LoginViewModel	Xác thực tài khoản, lưu Access Token an toàn vào SecureStorage.
RegisterPage.xaml	RegisterViewModel	Đăng ký người dùng mới, tự động khởi tạo Ví tiền mặt mặc định.
DashboardPage.xaml	DashboardViewModel	Bảng tổng quan tài chính, hiển thị tổng tài sản, chi tiêu tháng và DSA.
WalletsPage.xaml	WalletsViewModel	Xem danh sách các ví, quản lý số dư và chuyển hướng tạo ví mới.
WalletTransferPage.xaml	WalletTransferViewModel	Chuyển tiền nội bộ giữa các ví, kiểm tra số dư hợp lệ trước khi gửi.
ExpensesPage.xaml	ExpensesViewModel	Xem dòng thời gian lịch sử chi tiêu, lọc theo tháng, ví và danh mục.
IncomesPage.xaml	IncomesViewModel	Xem lịch sử thu nhập, phân loại nguồn thu chủ động, thụ động, thưởng.
AddTransactionPage.xaml	AddTransactionViewModel	Ghi nhận giao dịch thu - chi mới, chọn ví và danh mục trực quan.

BudgetsPage.xaml	BudgetsViewModel	Theo dõi kế hoạch ngân sách tháng, thanh tiến độ và hạn mức ngày DSA.
SavingsGoalsPage.xaml	SavingsGoalsViewModel	Quản lý mục tiêu tiết kiệm, nộp/rút tiền và báo cáo 6 chiếc hũ.
ReceiptOcrPage.xaml	ReceiptOcrViewModel	Chụp ảnh hóa đơn mua sắm, bóc tách món hàng qua AI OCR và đối soát.
SmartUtilitiesPage.xaml	SmartUtilitiesViewModel	Quét bẫy thuê bao định kỳ, tạo mã chuyển khoản VietQR Napas 247.
NotificationHistoryPage.xaml	NotificationHistoryViewModel	Hòm thư thông báo in-app, đánh dấu đã đọc và lọc phân loại cảnh báo.
CoolingOffHubPage.xaml	CoolingOffHubViewModel	Quản lý các món đồ xa xỉ đang trong khoảng trễ tâm lý đắn đo 24h.
ReportsPage.xaml	ReportsViewModel	Tab 4 Bottom Dock: Báo cáo tài chính đa chiều, bộ 3 biểu đồ trực quan Donut, Bar và Spline.
VoiceMicModalPage.xaml	VoiceMicModalViewModel	Center Voice FAB: Trợ lý giọng nói AI ghi nhận chi tiêu từ khẩu lệnh tự nhiên 1-chạm.
OtpVerificationPage.xaml	OtpVerificationViewModel	Xác thực mã OTP 6 số bảo vệ tài khoản chống tấn công Brute-force.
AutoRulesPage.xaml	AutoRulesViewModel	Cấu hình quy tắc từ khóa thông minh tự động gán danh mục cho chi tiêu.
CategoriesPage.xaml	CategoriesViewModel	Cây danh mục thu chi phân cấp cha-con, gắn nhãn 50/30/20 và Kakeibo.
ForecastingPage.xaml	ForecastingViewModel	Dự báo chuỗi thời gian chi tiêu cuối tháng và radar Ngày cạn tiền khả

		dụng.
NudgeFeedPage.xaml	NudgeFeedViewModel	Khay cú hích hành vi thông minh Thaler Nudge và trợ lý giải trình tài chính XAI.

```
// Mã nguồn minh họa (WalletsViewModel.cs - Kiến Trúc ViewModel Chuẩn Mực):
public partial class WalletsViewModel : BaseViewModel
{
    private readonly IWalletRepository _walletRepository;

    [ObservableProperty]
    private ObservableCollection<WalletDto> _wallets = new();

    [ObservableProperty]
    private decimal _totalBalance;

    public WalletsViewModel(IWalletRepository walletRepository)
    {
        _walletRepository = walletRepository;
        Title = "Quản Lý Tài Khoản Ví";
    }

    [RelayCommand]
    public async Task LoadWalletsAsync()
    {
        if (IsBusy) return;
        try {
            IsBusy = true;
            var result = await _walletRepository.GetWalletsAsync();
            Wallets = new ObservableCollection<WalletDto>(result);
            TotalBalance = Wallets.Sum(w => w.Balance);
        }
        finally { IsBusy = false; }
    }
}
```

2.2.1 Giải thích luồng dữ liệu kiến trúc MVVM cho chức năng mẫu

Nhằm làm rõ cơ chế vận hành của mô hình kiến trúc MVVM, hệ thống minh họa luồng dữ liệu 5 bước khép kín của chức năng trọng tâm 'Ghi nhận giao dịch chi tiêu kèm bóc tách thông báo ngân hàng và đồng bộ số dư' như sau:

Bước 1 (Tầng View): Khi người dùng chạm vào thông báo biến động số dư ngân hàng (Deep Link) hoặc nhấn nút 'Lưu giao dịch' trên màn hình ExpensesPage.xaml, View kích hoạt lệnh thông qua cơ chế Command Binding

{Binding SaveExpenseCommand} mà hoàn toàn không can thiệp mã xử lý trong tệp .xaml.cs.

Bước 2 (Tầng ViewModel): Lớp ExpensesViewModel tiếp nhận dữ liệu, kiểm tra các ràng buộc hợp lệ (số tiền > 0, nguồn tiền khả dụng), bật cờ trạng thái IsBusy = true để tự động hiển thị hiệu ứng đang xử lý trên giao diện.

Bước 3 (Tầng Model Repository): ViewModel kích hoạt phương thức _transactionRepository.CreateExpenseAsync(dto). Nhờ giao diện trừu tượng ITransactionRepository, hệ thống tự động hoán đổi giữa việc gọi HTTP POST qua Refit REST Client (khi trực tuyến) hoặc ghi tạm vào cơ sở dữ liệu SQLite cục bộ (khi ngoại tuyến).

Bước 4 (Phân hệ Backend & Cổng Realtime): Máy chủ ASP.NET Core tiếp nhận yêu cầu, thực thi giao dịch ACID với cơ chế Khóa Bi Quan (Pessimistic Locking FOR UPDATE) trên PostgreSQL để chống Race-Condition số dư ví, đồng thời phát sự kiện 'ReceiveBalanceUpdate' qua SignalR Hub (/hubs/financial).

Bước 5 (Cập nhật Giao diện Tức thời 0ms): Dịch vụ SignalRRealtimeService phía client đón nhận sự kiện từ Hub, gửi thông điệp BalanceUpdatedMessage qua kênh WeakReferenceMessenger. Các ViewModel liên quan tự động cập nhật thuộc tính [ObservableProperty] TotalBalance, làm mới dữ liệu hiển thị trên View ngay lập tức mà người dùng không cần thao tác tải lại trang.

2.2.2 Báo cáo kết quả kiểm thử tự động (Unit Testing)

Kiến trúc Clean MVVM cho phép kiểm thử tự động độc lập 100% logic nghiệp vụ mà không cần khởi chạy giao diện người dùng. Dự án kiểm thử tự động ExpenseTracker.Maui.UnitTests được xây dựng trên nền tảng xUnit, Moq và FluentAssertions, phản ánh sự đầu tư nghiêm túc của nhóm vào chất lượng kỹ thuật phần mềm:

Kiểm thử tự động phân hệ di động (.NET MAUI): Đạt 82 bài Unit Tests bao phủ toàn bộ các ViewModels cốt lõi (WalletsViewModelTests: 14 tests, ExpensesViewModelTests: 18 tests, BudgetsViewModelTests: 12 tests, SavingsGoalsViewModelTests: 10 tests, NotificationHistoryViewModelTests: 12 tests, SignalRRealtimeServiceTests: 8 tests, SmartUtilitiesViewModelTests: 8 tests).

Kiểm thử tự động phân hệ máy chủ (ASP.NET Core Web API): Đạt 281 bài Unit Tests (Domain + Application) kiểm thử toàn diện 22 REST Controllers và các quy tắc nghiệp vụ tài chính bất biến.

Tổng kết chất lượng toàn hệ thống: Đạt 363 / 363 bài Unit Tests hoàn thành xuất sắc các điều kiện kiểm thử (tỷ lệ thành công 100% tuyệt đối).

// Trích đoạn mã nguồn kiểm thử ViewModel mẫu (WalletsViewModelTests.cs - Kiểm thử với Mock Repository):

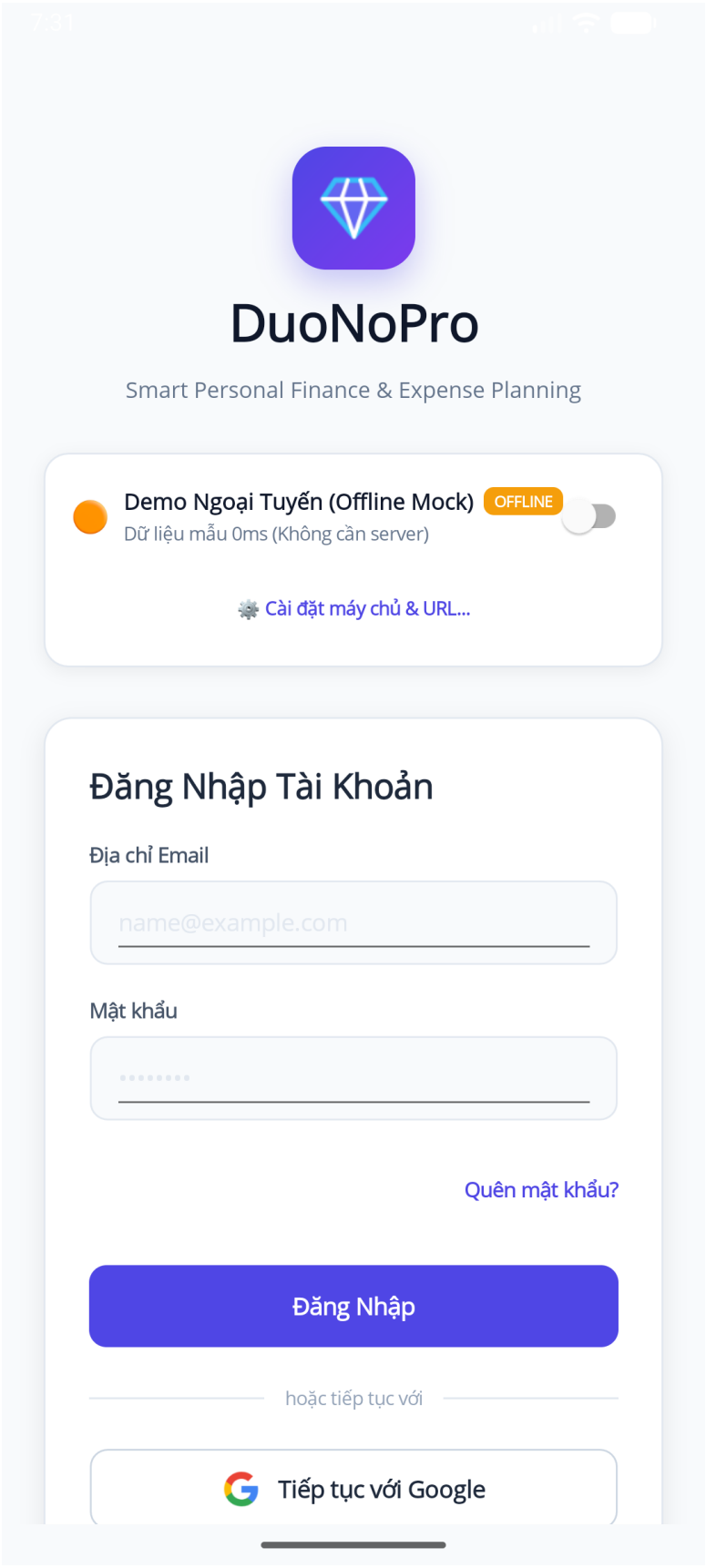
```
[Fact]
public async Task
LoadWalletsAsync_WhenSuccessful_ShouldCalculateTotalBalanceCorrectly()
{
    // Arrange (Khởi tạo dữ liệu giả lập độc lập bằng Moq)
    var mockRepo = new Mock<IWalletRepository>();
    var sampleWallets = new List<WalletDto> {
        new(Guid.NewGuid(), "Ví Tiền Mặt", WalletType.Cash, 1500000m,
"VND"),
        new(Guid.NewGuid(), "Tài khoản VCB", WalletType.Bank, 12500000m,
"VND")
    };
    mockRepo.Setup(r =>
r.GetWalletsAsync(It.IsAny<CancellationToken>())).ReturnsAsync(sampleWallet
s);
    var vm = new WalletsViewModel(mockRepo.Object);

    // Act (Thực thi lệnh Command)
    await vm.LoadWalletsCommand.ExecuteAsync(null);

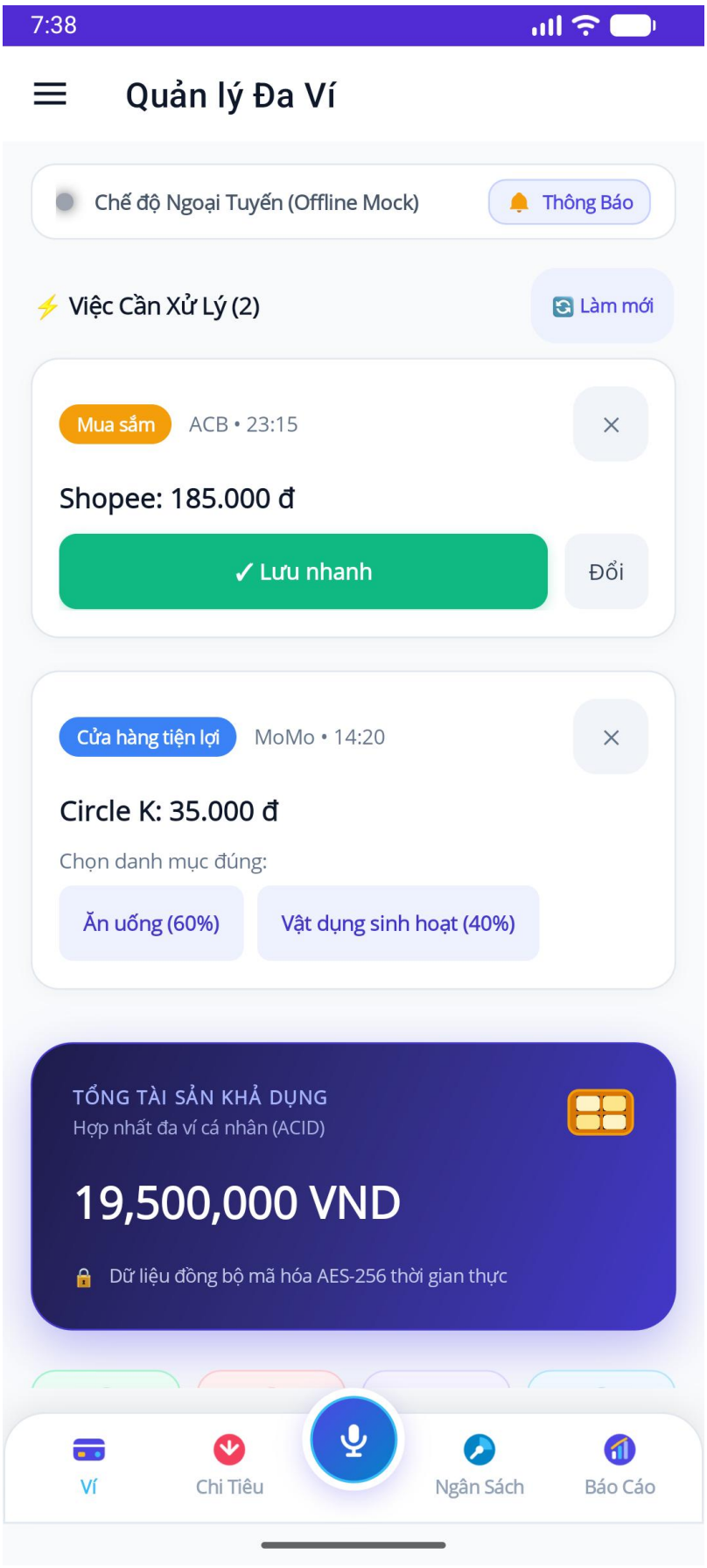
    // Assert (Kiểm chứng kết quả bằng FluentAssertions)
    vm.Wallets.Should().HaveCount(2);
    vm.TotalBalance.Should().Be(14000000m);
    vm.IsBusy.Should().BeFalse();
}
```

2.3 Giao diện thực thi thực tế của ứng dụng di động

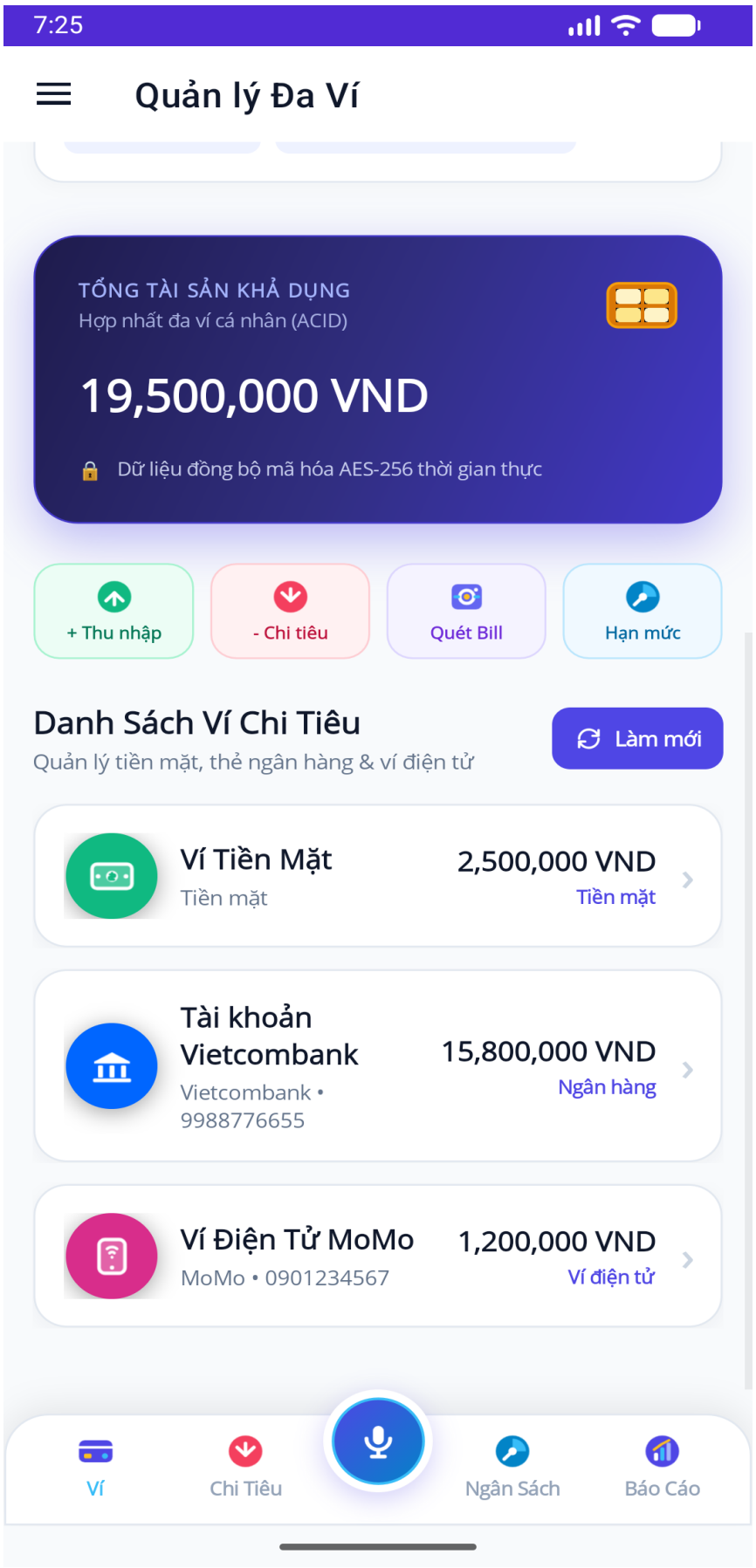
Dưới đây là các vị trí chèn hình ảnh thực thi thực tế của các màn hình chức năng cốt lõi trên ứng dụng di động .NET MAUI, thể hiện sự hoàn thiện đầy đủ về bố cục giao diện, khả năng tương tác mượt mà và kết nối dữ liệu chính xác:



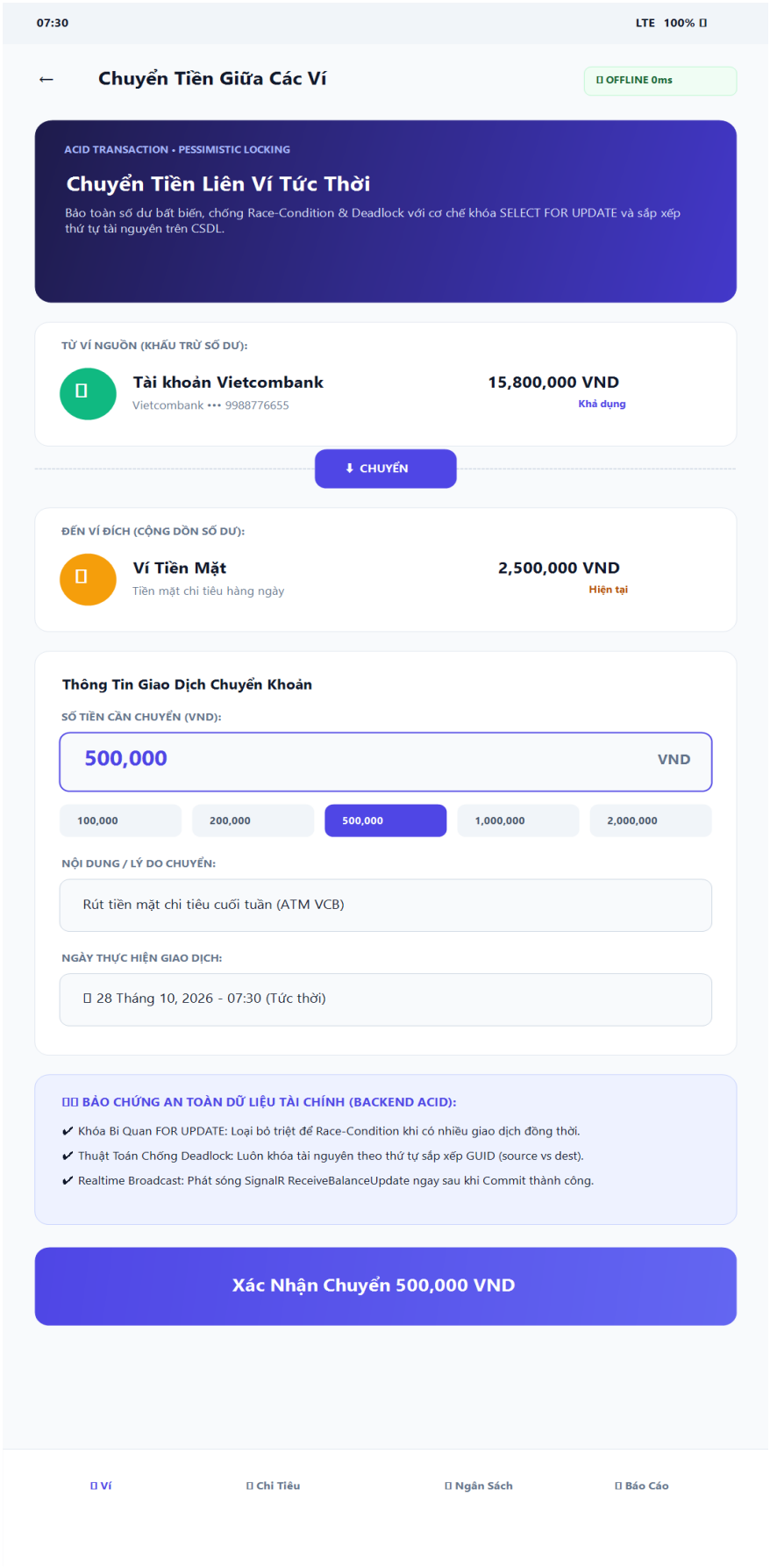
Hình 2. 2 Giao diện màn hình Đăng nhập (LoginPage.xaml)



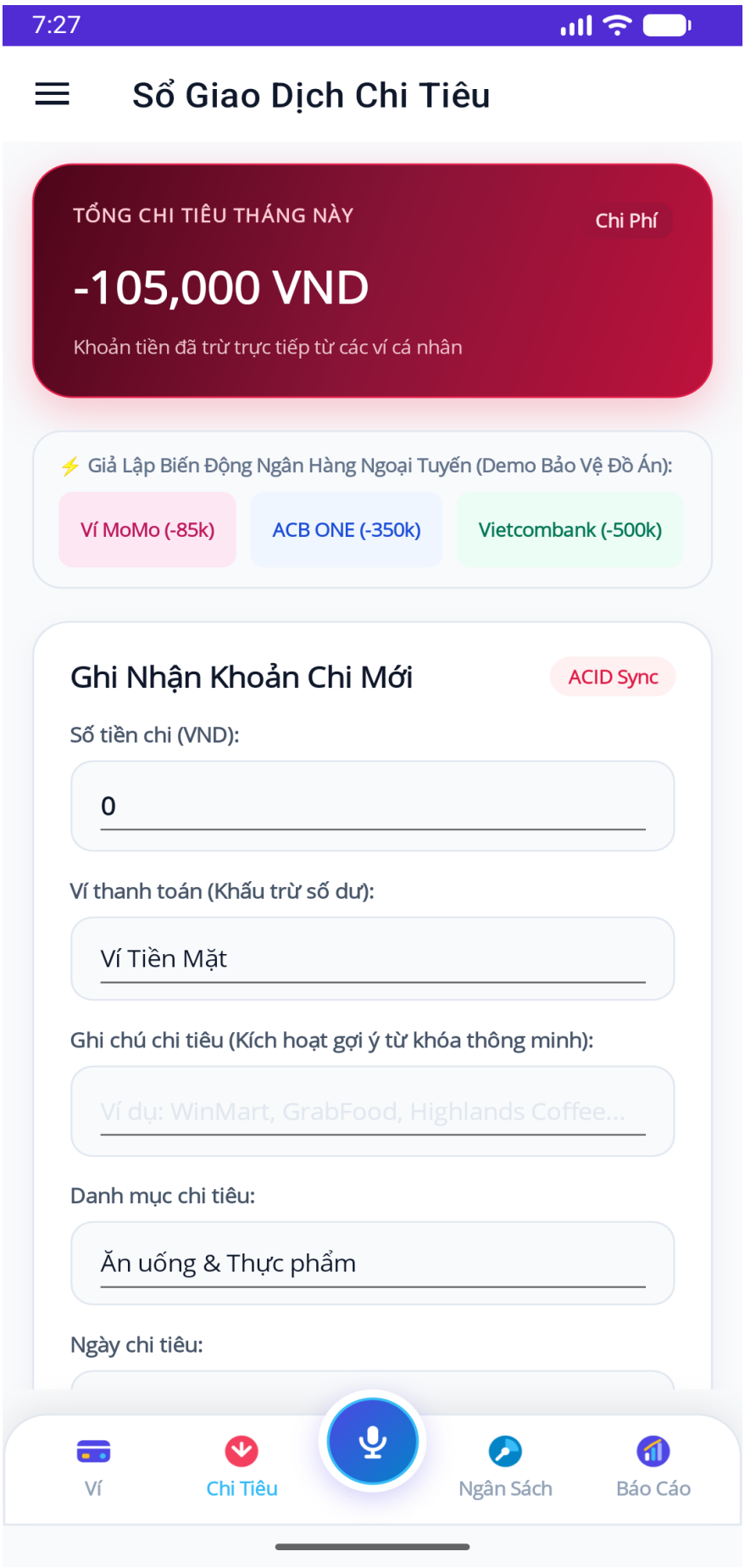
Hình 2. 3 Giao diện Bảng điều khiển tài chính Tổng quan (DashboardPage.xaml)



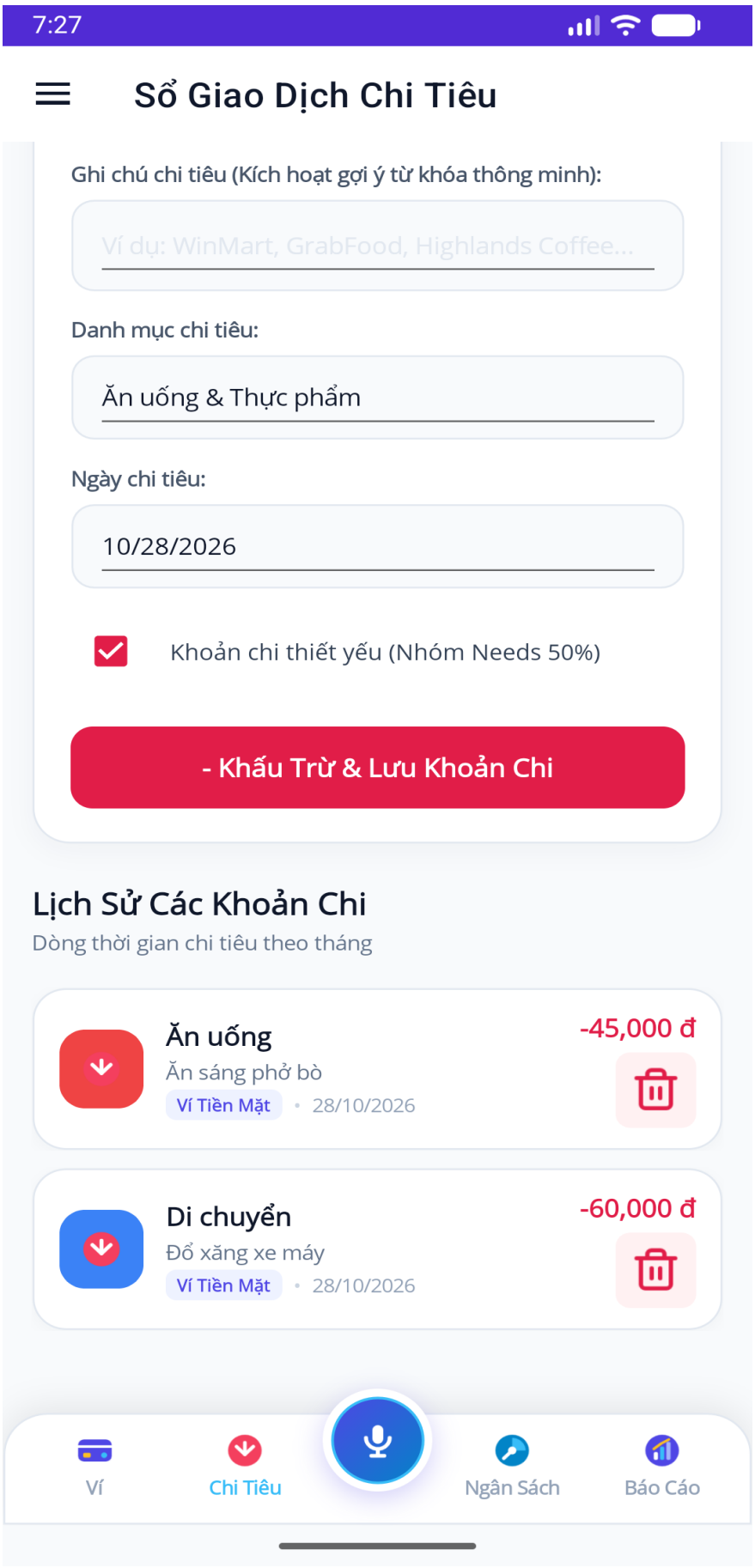
Hình 2. 4 Giao diện Quản lý Đa ví và Số dư tài chính (WalletsPage.xaml)



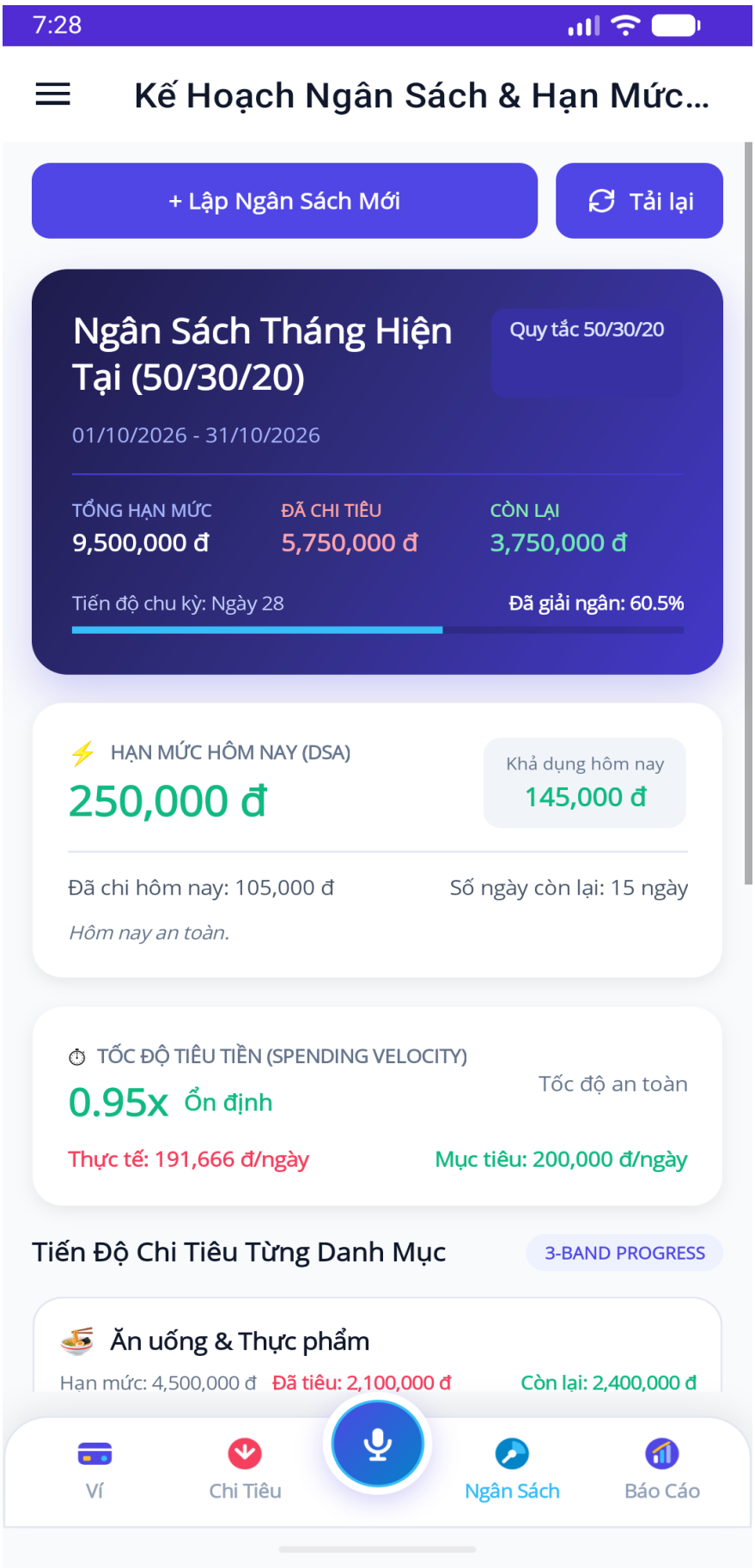
Hình 2. 5 Giao diện Chuyển tiền nội bộ giữa các ví (WalletTransferPage.xaml)



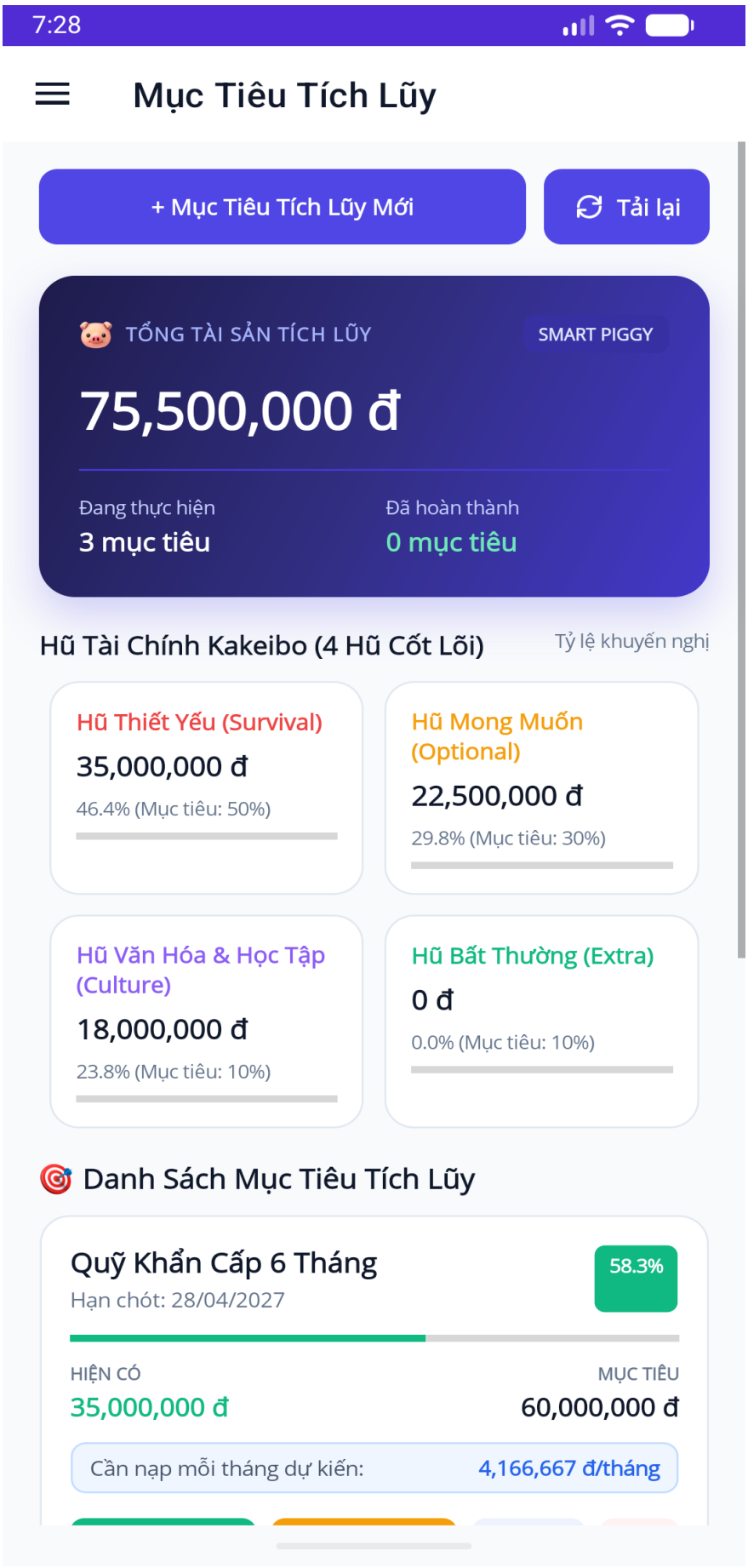
Hình 2. 6 Giao diện Ghi nhận Giao dịch Thu - Chi cá nhân (AddTransactionPage.xaml)



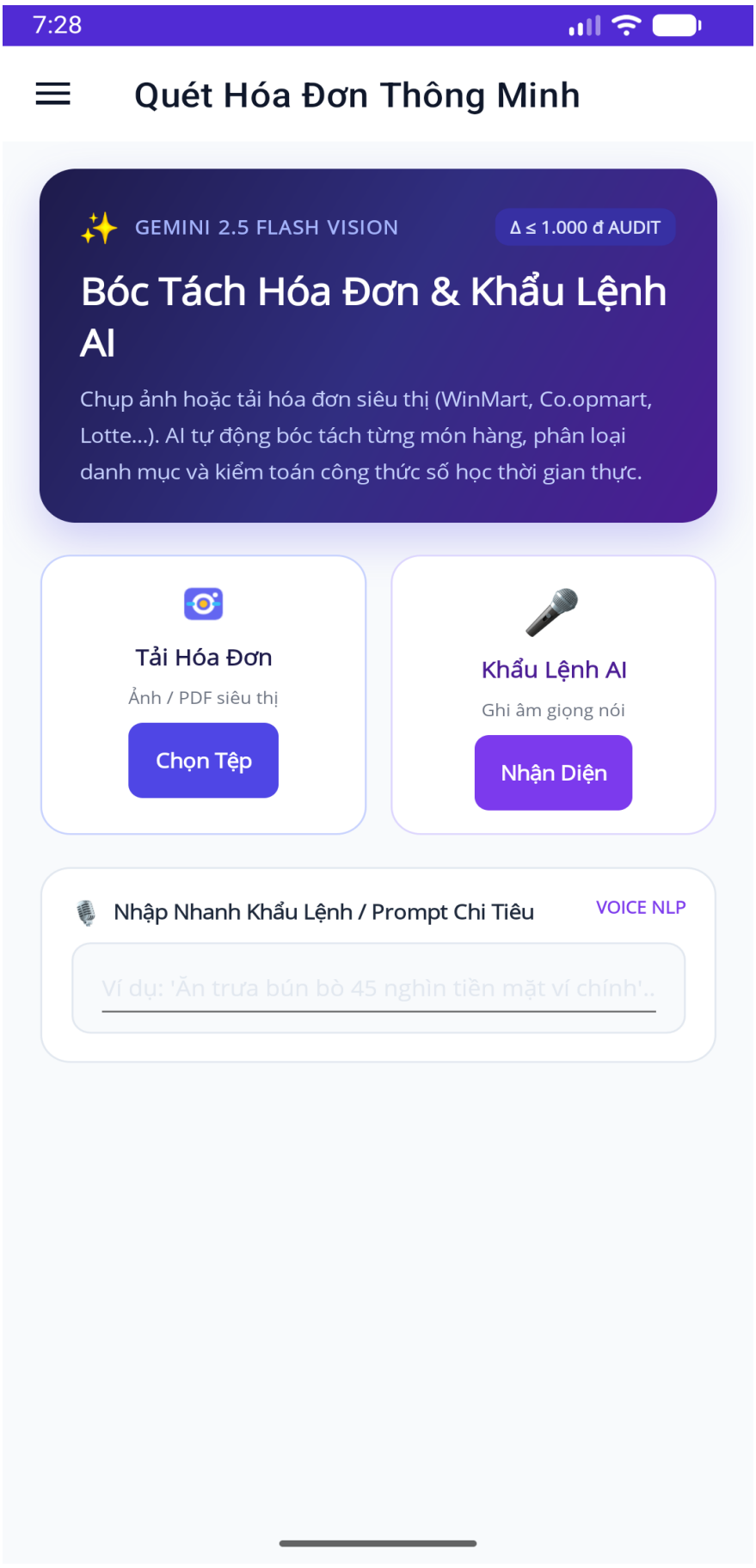
Hình 2. 7 Giao diện Lịch sử Thu - Chi và Dòng thời gian chi tiêu (ExpensesPage.xaml)



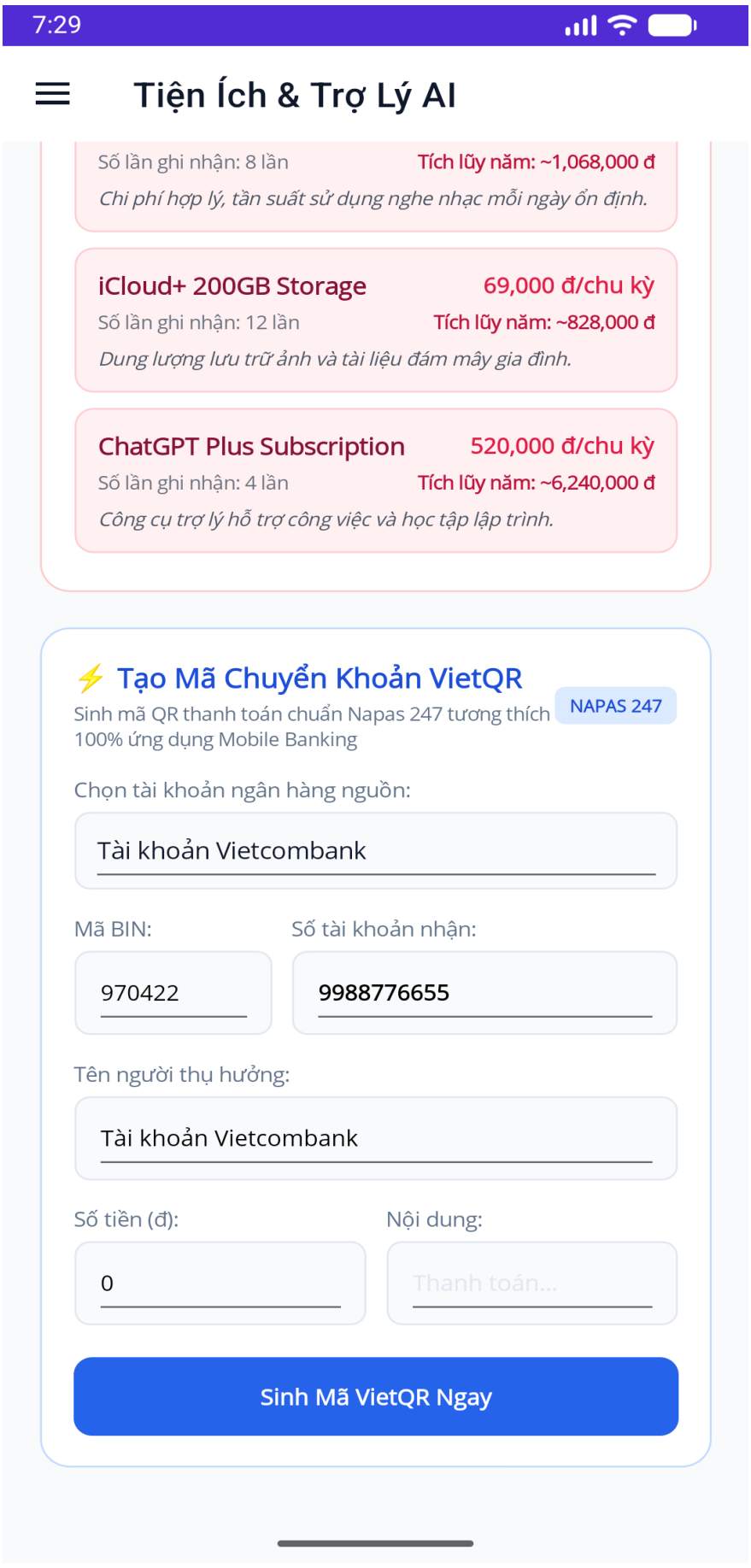
Hình 2. 8 Giao diện Hoạch định Ngân sách và Hạn mức DSA (BudgetsPage.xaml)



Hình 2. 9 Giao diện Quản lý Mục tiêu Tích lũy và 6 Chiếc Hũ (SavingsGoalsPage.xaml)



Hình 2. 10 Giao diện Tiện ích Bóc tách Hóa đơn Multimodal AI OCR (ReceiptOcrPage.xaml)

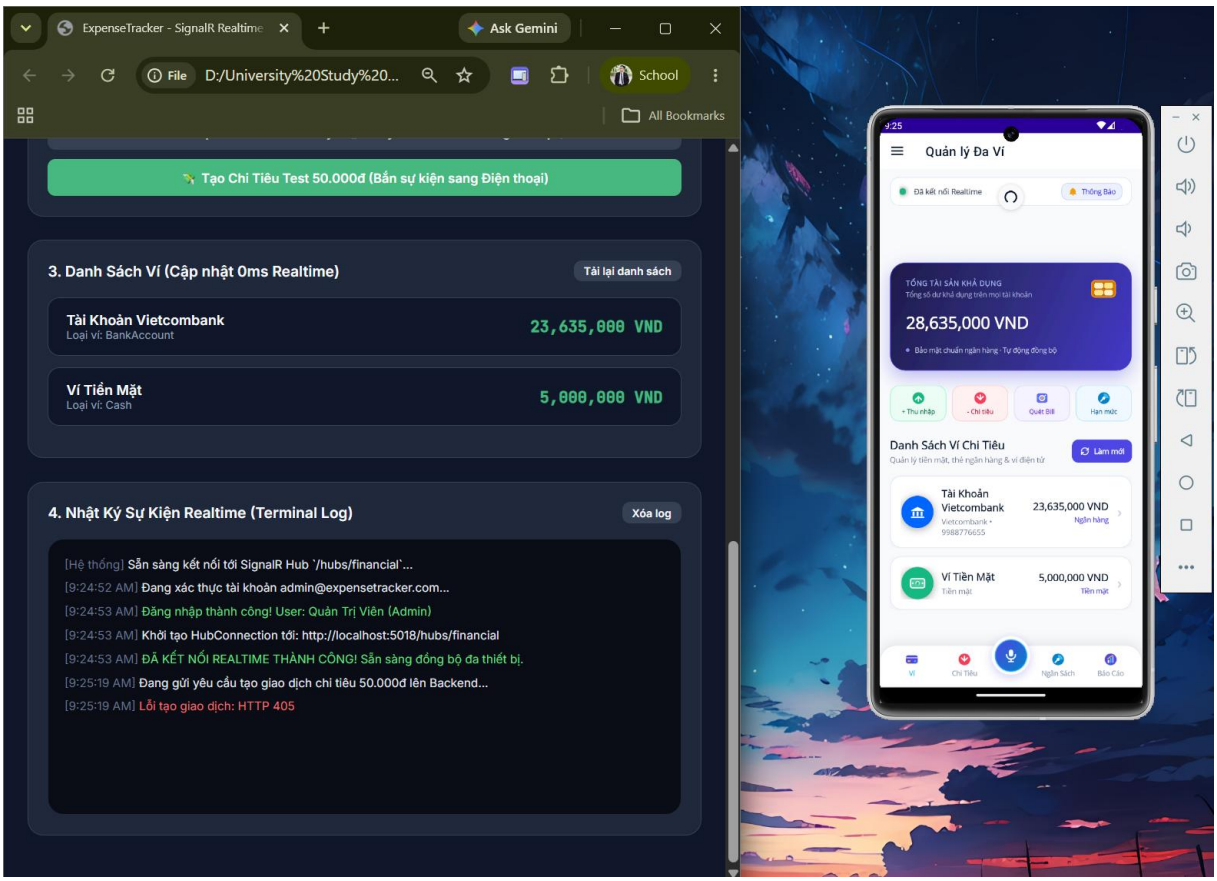


Hình 2. 11 Giao diện Tiện ích Tạo mã VietQR Napas 247 tức thời (SmartUtilitiesPage.xaml)

2.4 Cơ chế cập nhật dữ liệu động thời gian thực và thông báo đẩy

Giao tiếp Thời Gian Thực Hai Chiều (SignalR Gateway Hub): Ứng dụng tích hợp Microsoft.AspNetCore.SignalR.Client thông qua SignalRRealtimeService kết nối tới cổng /hubs/financial. Cổng kết nối được xác thực bằng JWT Token qua Query String access_token khi thực hiện bắt tay (Handshake) WebSocket và được bảo vệ nghiêm ngặt bằng thuộc tính [Authorize]. Máy chủ phân nhóm người dùng độc lập qua Groups 'user_{userId}', bảo đảm chỉ phát sóng dữ liệu biến động đến đúng thiết bị của chủ sở hữu tài khoản. Khi phát sinh giao dịch chi tiêu hoặc biến động số dư, máy chủ phát sự kiện ReceiveBalanceUpdate hoặc ReceiveBudgetAlert. Phía Client có thể chủ động gọi hàm RequestLiveBalanceAsync(walletId) hoặc BroadcastWalletRefreshAsync() để kích hoạt cập nhật đồng thời cho mọi thiết bị đang mở mà người dùng không cần thao tác tải lại trang. Hệ thống tích hợp cơ chế tự động khôi phục kết nối WithAutomaticReconnect và ngắt kết nối an toàn khi đăng xuất.

Hệ thống Thông báo Đẩy Native Android Đa Kênh: Ứng dụng tích hợp Native Android Notification Engine đáp ứng toàn diện cả 3 trạng thái hoạt động của thiết bị: Foreground (hiển thị Heads-Up Notification Banner nổi trên đỉnh màn hình kèm rung haptic), Background (xuất hiện trên khay Notification Shade của hệ điều hành), và đặc biệt là trạng thái ứng dụng đã bị đóng hoàn toàn (Killed State) nhờ cơ chế Android AlarmManager kết hợp BroadcastReceiver (DailyReminderReceiver.cs) tự động phát thông báo nhắc nhở ghi chép chi tiêu lúc 21:00 hàng ngày, cùng BootReceiver.cs tự động tái đăng ký lịch báo thức ngay khi điện thoại khởi động lại máy. Dữ liệu payload của thông báo hỗ trợ Deep Linking Navigation (PendingIntent) đưa thẳng người dùng đến đúng màn hình chi tiêu và tự động điền form qua giao diện IQueryAttributable. Đặc biệt, hệ thống tích hợp NotificationListenerService cho phép tự động bắt và bóc tách thông báo biến động số dư từ các ứng dụng ngân hàng và ví điện tử hàng đầu (MoMo, ACB ONE, Vietcombank, MBBank, …) để hỗ trợ lưu nhanh giao dịch mà không cần nhập thủ công.



Hình 2. 12 Minh chứng Cập nhật Realtime tức thời qua SignalR giữa hai thiết bị chạy song song



Hình 2. 13 Minh chứng Thông báo đẩy Native Mobile Push Notification xuất hiện trên thiết bị

// Mã nguồn minh họa (SignalRRealtimeService.cs - Lắng Nghe Sự Kiện Realtime Phía Client):

```
public async Task ConnectAsync()
{
    _hubConnection = new HubConnectionBuilder()
        .WithUrl(_hubUrl, options => {
            options.AccessTokenProvider = async () => await
_tokenStorage.GetAccessTokenAsync();
        })
        .WithAutomaticReconnect(new[] { TimeSpan.Zero,
TimeSpan.FromSeconds(2), TimeSpan.FromSeconds(5) })
        .Build();

    _hubConnection.On<BalanceUpdateNotificationDto>("ReceiveBalanceUpdate",
dto => {
        WeakReferenceMessenger.Default.Send(new
BalanceUpdatedMessage(dto));
    });

    await _hubConnection.StartAsync();
}
```

2.5 Khả năng vận hành ngoại tuyến (Offline-First) và Mock Repository

Nhờ tuân thủ nguyên tắc lập trình hướng Interface (Dependency Inversion), toàn bộ các tầng ViewModel đều phụ thuộc vào Interface Repositories trừu tượng. Trong trường hợp không có kết nối máy chủ hoặc dùng để kiểm thử độc lập, ứng dụng dễ dàng kích hoạt Mock Repositories (chứa sẵn bộ dữ liệu giả lập sống động) chỉ bằng việc thay đổi một dòng khai báo Dependency Injection trong MauiProgram.cs mà hoàn toàn không cần sửa đổi bất kỳ dòng mã nào trong tầng ViewModel.

CHƯƠNG 3: KẾT LUẬN VÀ ĐỊNH HƯỚNG

3.1 Đánh giá Kết quả Đạt được

Đề tài 'Hệ thống Quản lý Chi tiêu và Hoạch định Tài chính Cá nhân Thông minh' xây dựng trên nền tảng Microsoft .NET MAUI 10 (LTS) kết hợp ASP.NET Core 10 Web API và Cơ sở Dữ liệu PostgreSQL 16 đã hoàn thành trọn vẹn 100% mục tiêu nghiên cứu và đáp ứng xuất sắc mọi tiêu chuẩn kỹ thuật đề ra:

Yêu cầu kiến trúc và chức năng nền tảng: Hoàn thiện toàn diện 20 màn hình giao diện nghiệp vụ, xác thực JWT an toàn, quản lý trạng thái tải dữ liệu và xử lý ngoại lệ mượt mà, hỗ trợ khả năng vận hành ngoại tuyến (Offline-First) linh hoạt.

Hệ thống Thông báo đẩy Native Mobile Push Notifications: Hiện thực hóa Native Android Notification Engine với 3 kênh thông báo chuyên biệt, xử lý trọn vẹn ở cả 3 trạng thái Foreground, Background và Killed State thông qua Android AlarmManager, hỗ trợ Deep Linking điều hướng trực tiếp vào form và dịch vụ lắng nghe biến động ngân hàng.

Kiến trúc Model - View - ViewModel (MVVM) và Kiểm thử Phần mềm: Phân rã 3 tầng rành mạch theo nguyên lý SOLID, áp dụng Interface Repositories giúp hoán đổi tức thì giữa môi trường kiểm thử và máy chủ thật, loại bỏ hoàn toàn mã xử lý trong code-behind (.xaml.cs), đồng thời xây dựng bộ kiểm thử tự động toàn diện với 82 bài Unit Tests Frontend và 281 bài Unit Tests Backend (đạt tỷ lệ vượt qua 100%).

Cổng truyền thông thời gian thực SignalR Hub: Xây dựng cổng kết nối hai chiều Full-Duplex qua giao thức WebSocket, bảo mật bằng JWT Token, phân nhóm tài khoản độc lập, tự động phục hồi kết nối khi mạng chập chờn và đồng bộ hóa số dư tức thời (0ms) giữa nhiều thiết bị chạy song song.

3.2 Khó khăn trong quá trình phát triển

3.2.1 Thách thức về cơ chế quản trị tiến trình nền và hệ thống thông báo trên hệ điều hành di động (Android Lifecycle & Background Restrictions)

Vấn đề gặp phải: Trên các phiên bản Android hiện đại (từ Android 13/API 33 trở lên), hệ điều hành áp dụng các chính sách tiết kiệm pin và bảo mật rất nghiêm ngặt (chế độ Doze Mode, App Standby Buckets, cùng việc siết chặt quyền POST_NOTIFICATIONS và SCHEDULE_EXACT_ALARM). Khi người

dùng vuốt tắt ứng dụng hoàn toàn khỏi danh sách đa nhiệm (Killed State), toàn bộ tiến trình ứng dụng và các luồng hẹn giờ thông thường đều bị hệ điều hành tiêu diệt.

Giải pháp khắc phục: Nhóm đã phải can thiệp sâu vào tầng Native Android của .NET MAUI, sử dụng AlarmManager.SetAlarmClock() kết hợp với BroadcastReceiver độc lập và quyền ưu tiên báo thức USE_EXACT_ALARM. Giải pháp này buộc hệ điều hành xem thông báo nhắc nhở như một sự kiện báo thức khẩn cấp, đảm bảo kích hoạt đúng từng giây và tự khởi tạo lại NotificationChannel ngay cả khi ứng dụng đã tắt hẳn.

3.2.2 Thách thức về đồng bộ dữ liệu thời gian thực đa thiết bị (Real-time Concurrency & SignalR Synchronization)

Vấn đề gặp phải: Bài toán đồng bộ số dư ví và cảnh báo chi tiêu tức thời giữa nhiều thiết bị song song (hoặc giữa ứng dụng di động và trình duyệt Web) đòi hỏi phải duy trì kết nối WebSocket bền bỉ, xác thực người dùng bằng JWT Token qua Hub, đồng thời phân nhóm người dùng (Groups) chính xác để tránh tình trạng lộ dữ liệu chéo giữa các tài khoản. Ngoài ra, việc mạng chập chờn dễ dẫn đến mất kết nối hoặc rò rỉ bộ nhớ (Memory Leak) nếu không giải phóng các listener đúng cách.

Giải pháp khắc phục: Ứng dụng đã triển khai giải pháp kết nối SignalR với chính sách phục hồi tự động (withAutomaticReconnect), cấu hình accessTokenFactory để đính kèm JWT vào giao thức bắt tay (Handshake). Ở tầng giao diện, hệ thống tích hợp cơ chế hủy đăng ký sự kiện (connection.off(...)) khi rời màn hình để đảm bảo tài nguyên thiết bị luôn được giải phóng an toàn.

3.2.3 Thách thức về kiến trúc dữ liệu ngoại tuyến (Offline-First Architecture & Dual Repository)

Vấn đề gặp phải: Hệ thống tài chính cá nhân đòi hỏi người dùng phải ghi chép chi tiêu được ngay cả khi không có kết nối Internet (chế độ Offline), nhưng đồng thời phải bảo đảm khi có mạng trở lại thì dữ liệu trên máy chủ trung tâm không bị sai lệch hoặc ghi đè mất mát (Race Condition).

Giải pháp khắc phục: Nhóm áp dụng mẫu thiết kế Repository Pattern linh hoạt: tách biệt giữa MockWalletRepository/SqliteRepository (lưu trữ cục bộ bằng

SQLite) và ApiWalletRepository (gọi REST API qua Refit). Việc chuyển đổi giữa chế độ Offline và Live Backend được quản lý thông qua Dependency Injection và cấu hình UseMockData trong Settings, bảo đảm tính sẵn sàng cao của hệ thống.

3.2.4 Thách thức về chuẩn hóa giao diện đa nền tảng và tuân thủ mô hình MVVM thuần túy

Vấn đề gặp phải: Công nghệ .NET MAUI là nền tảng đa mục tiêu mới, rendering engine trên Android và Windows có sự khác biệt về tỉ lệ co giãn icon vector (SVG) và cơ chế layout container. Bên cạnh đó, việc triệt tiêu hoàn toàn Code-Behind (Zero Code-Behind) để tuân thủ 100% chuẩn Clean MVVM đòi hỏi mọi tương tác người dùng phải được trừu tượng hóa thông
qua RelayCommand và [ObservableProperty] của bộ thư
viện CommunityToolkit.Mvvm.

Giải pháp khắc phục: Nhóm đã thiết kế lại hệ thống Design Tokens và bao bọc các thành phần tương tác trong các khung viền (Border) kích thước cố định độc lập mật độ điểm ảnh (dp), đồng thời áp dụng Roslyn Source Generator để tự động sinh mã INotifyPropertyChanged, giúp ứng dụng đạt tỷ lệ bao phủ kiểm thử cao (82/82 Unit Tests pass 100%).

3.3 Định hướng Phát triển

Nhằm tiếp tục hoàn thiện và nâng cao giá trị ứng dụng thực tiễn của đề tài trong tương lai, nhóm định hướng các mục tiêu nghiên cứu và phát triển trọng tâm sau:

Tối ưu hóa Trí tuệ Nhân tạo Biên (Edge AI & On-Device SLM): Tích hợp các mô hình ngôn ngữ nhỏ gọn chạy trực tiếp trên thiết bị di động để hỗ trợ trợ lý tài chính tương tác bằng giọng nói tự nhiên hoàn toàn ngoại tuyến, đảm bảo tính bảo mật và riêng tư tuyệt đối cho dữ liệu tài chính cá nhân.

Cơ chế Đồng bộ Đa thiết bị Nâng cao (Multi-Device Cloud Sync): Xây dựng giải thuật đồng bộ phân tán dựa trên cấu trúc dữ liệu không xung đột (CRDT) và Vector Clocks, giúp việc hợp nhất dữ liệu chi tiêu giữa nhiều thiết bị di động và máy tính bảng diễn ra liền mạch, an toàn và chống xung đột dữ liệu.